

React Native từ cơ bản đến nâng cao

— Tập 1: Cơ bản

Cài đặt, component, styling, Flexbox, danh sách, form, điều hướng — và React Native cho Angular developer

Expo SDK 57 · React Native 0.86.3 · React 19.2.3 · TypeScript 6.0.3

Bản dựng: 2026-09-28 — Markdown là nguồn chính (books/react-native/vol1-co-ban/)

Kiểm tra dấu tiếng Việt: ă ằ ẳ ẵ ẫ ơ ư đ Ư Đ — Đường ướt, trắng khuyết, Ưu tiên

Mục lục

Chương 1 — Cài đặt và dự án đầu tiên

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 2 — React Native cho Angular developer

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 3 — Component, JSX, props, state và các core component

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 4 — Styling và Flexbox

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 5 — Danh sách: FlatList và SectionList

Mục tiêu

[Giải thích đơn giản](#)

[Ví dụ](#)

[Đi sâu](#)

[Lỗi và bẫy thường gặp](#)

[Tóm tắt](#)

[Bài tập \(có lời giải\)](#)

[Nguồn tham khảo \(Sources\)](#)

[Chương 6 — Form và nhập liệu](#)

[Mục tiêu](#)

[Giải thích đơn giản](#)

[Ví dụ](#)

[Đi sâu](#)

[Lỗi và bẫy thường gặp](#)

[Tóm tắt](#)

[Bài tập \(có lời giải\)](#)

[Nguồn tham khảo \(Sources\)](#)

[Chương 7 — Điều hướng \(navigation\) với Expo Router](#)

[Mục tiêu](#)

[Giải thích đơn giản](#)

[Ví dụ](#)

[Đi sâu](#)

[Lỗi và bẫy thường gặp](#)

[Tóm tắt](#)

[Bài tập \(có lời giải\)](#)

[Nguồn tham khảo \(Sources\)](#)

[Chương 8 — App mẫu Tập 1: "Việc Cần Làm"](#)

[Mục tiêu](#)

[Giải thích đơn giản](#)

[Ví dụ](#)

[Đi sâu](#)

[Lỗi và bẫy thường gặp](#)

[Tóm tắt](#)

[Bài tập \(có lời giải\)](#)

[Nguồn tham khảo \(Sources\)](#)

[STACK — Phiên bản và thư viện dùng cho cả bộ sách](#)

[1. Tóm tắt nhanh \(TL;DR\)](#)

[2. Phiên bản React Native mới nhất và New Architecture](#)

[3. Expo hay React Native CLI?](#)

[4. Expo SDK hiện tại và Expo Go trên iPhone](#)

[5. Thư viện cho từng tập \(tất cả có trong Expo Go SDK 57\)](#)

6. Công cụ trong sandbox (dùng để kiểm tra sách)

Nguồn tham khảo (Sources)

GLOSSARY — Thuật ngữ React Native (Anh → Việt)

Chương 1 — Cài đặt và dự án đầu tiên

Mục tiêu

Sau chương này bạn sẽ:

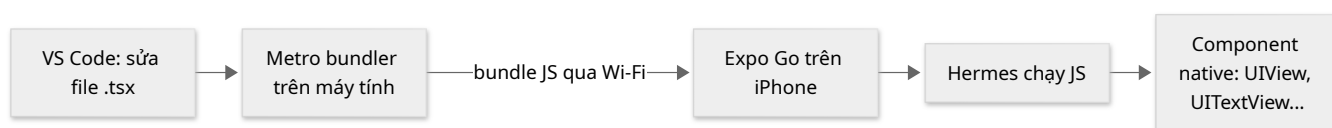
- Hiểu React Native (RN) là gì và Expo đứng ở đâu.
- Tạo một dự án Expo SDK 57 bằng TypeScript.
- Chạy app trên iPhone bằng **Expo Go** và thấy **Fast Refresh** (tự cập nhật khi lưu file).
- Biết cách chạy type-check, lint và test cho dự án.

Giải thích đơn giản

React Native cho phép bạn viết app iOS/Android bằng TypeScript + React. Khác với Ionic (chạy trong WebView), RN vẽ **component native thật** (native component): `<View>` trên iOS là `UIView`, `<Text>` là `UITextView` (theo bảng trong tài liệu RN). Code TypeScript của bạn chạy trong một **JavaScript engine** tên là **Hermes** trên điện thoại.

Expo là một **framework** (bộ khung) cho React Native — giống như Angular CLI + Angular Router + một thư viện lớn các module native (camera, vị trí, lưu trữ...). Tài liệu chính thức của React Native khuyên: "if you're building a new app with React Native, we recommend using a Framework", và Expo là framework được giới thiệu đầu tiên.

Expo Go là một app có sẵn trên App Store. Nó chứa sẵn phần native của Expo SDK. Máy tính của bạn chạy **Metro** (bundler — công cụ đóng gói JS, giống webpack/esbuild của Angular CLI); Expo Go tải bundle JS qua mạng LAN và chạy nó. Vì vậy bạn **không cần Mac hay Xcode** để thử app trên iPhone.



Ví dụ

Bước 1 — Chuẩn bị

Công cụ	Phiên bản dùng trong sách	Ghi chú
Node.js	22.22.2 (LTS)	RN 0.85+ cần Node $\geq$ 20.19.4
Expo SDK	57 (expo@57.0.25)	Xem <code>STACK.md</code>
Expo Go (iPhone)	bản trên App Store hỗ trợ SDK 57	UNVERIFIED : xem <code>STACK.md</code> mục 4
Tài khoản Expo	miễn phí, tạo tại expo.dev	Bắt buộc cho Expo Go trên iPhone thật

Điều mới ở SDK 57: theo tài liệu Expo (`get-started/start-developing.mdx`), trên iPhone thật Expo Go chỉ mở project khi **Expo CLI** và **Expo Go** **đăng nhập cùng một tài khoản**.

Bước 2 — Tạo dự án

```
npx create-expo-app@latest hello-rn --template blank-typescript
cd hello-rn
npx expo login           # đăng nhập tài khoản Expo trên máy tính
npx expo start           # chạy Metro, in ra mã QR
```

Trên iPhone: mở Expo Go → đăng nhập **cùng tài khoản** (biểu tượng tài khoản góc trên bên phải) → quét mã QR bằng app Camera. iPhone và máy tính phải cùng mạng Wi-Fi.

Nếu mạng công ty chặn kết nối LAN, thử `npx expo start --tunnel`.

Trong sách này, mọi ví dụ Tập 1 nằm trong **một dự án**: `vol1-co-ban/examples/`.

```
cd books/react-native/vol1-co-ban/examples
npm ci
npx expo start
```

Mở tab **Lab** → "Ch.1 — Hello Expo".

Bước 3 — Component đầu tiên

File `examples/src/chapters/ch01/HelloExpo.tsx`:

```
import { useState } from 'react';
import { Platform, Pressable, StyleSheet, Text, View } from 'react-native';

export function HelloExpo() {
  const [count, setCount] = useState(0);

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Xin chào React Native 🙌</Text>
      <Text>Bạn đang chạy trên: {Platform.OS}</Text>
      <Pressable accessibilityRole="button" style={styles.button} onPress={() => setCount((c) => c + 1)}>
        <Text style={styles.buttonText}>Đã bấm {count} lần</Text>
      </Pressable>
    </View>
  );
}
```

Đổi chữ "Xin chào" rồi lưu: Expo Go cập nhật ngay mà không mất state (Fast Refresh).

Bước 4 — Kiểm tra chất lượng

```
npm run typecheck    # tsc --noEmit
npm run lint         # eslint với eslint-config-expo
npm test             # jest với preset jest-expo
```

Test cho component trên (`HelloExpo.test.tsx`):

```
test('bấm nút thì tăng bộ đếm', async () => {
  const user = userEvent.setup();
  await render(<HelloExpo />);
  expect(screen.getByText('Bạn đang chạy trên: ios')).toBeOnTheScreen(); // jest-expo mặc định giả lập iOS
  await user.press(screen.getByRole('button', { name: 'Đã bấm 0 lần' }));
  expect(screen.getByRole('button', { name: 'Đã bấm 1 lần' })).toBeOnTheScreen();
});
```

Kết quả thật (chạy ngày 2026-09-28, `npx jest --verbose src/chapters/ch01`):

```
PASS src/chapters/ch01/HelloExpo.test.tsx
  ✓ bấm nút thì tăng bộ đếm
PASS src/chapters/ch01/exercise.solution.test.tsx
  ✓ nút Đặt lại bị vô hiệu khi 0 và đưa bộ đếm về 0
Test Suites: 2 passed, 2 total
Tests:       2 passed, 2 total
```

Chạy trên Expo Go: NOT RUN (không có iPhone trong sandbox). Thay vào đó, chúng tôi đã chạy `npx expo export --platform ios` thành công: Metro tạo được bundle Hermes (`entry-....hbc`, khoảng 2.4 MB) cho iOS.

Đi sâu

Expo Go hay development build?

	Expo Go	Development build
Cài đặt	Tải từ App Store	Bạn tự build (EAS Build hoặc Xcode)
Native module	Chỉ các module có sẵn trong Expo SDK	Bất kỳ thư viện native nào
SDK	Chỉ một SDK (mới nhất)	SDK bạn chọn
Khi nào dùng	Học, prototype	App thật, sản phẩm

Tài liệu Expo (`develop/development-builds/introduction.md`) mô tả development build là "essentially your own version of Expo Go". Cả Tập 1 và Tập 2 chỉ dùng module có trong Expo Go. Tập 3 sẽ dùng development build.

New Architecture và Hermes (giới thiệu)

Từ React Native 0.82, app **chỉ** chạy trên **New Architecture** (kiến trúc mới: JSI, Fabric, TurboModules). Từ 0.84, **Hermes V1** là engine mặc định. Bạn không cần cấu hình gì; Tập 3 sẽ giải thích chi tiết.

Cấu trúc dự án ví dụ

```
examples/  
  app.json           # cấu hình app (tên, scheme, plugin) – giống angular.json + manifest  
  package.json       # phiên bản được ghim chính xác  
  tsconfig.json      # extends "expo/tsconfig.base", alias "@/*" → src/*  
  eslint.config.js   # eslint-config-expo (flat config)  
  src/app/           # các màn hình (Expo Router, file-based) – Chương 7  
  src/features/tasks/ # logic + component của app mẫu – Chương 8  
  src/chapters/chNN/ # code ví dụ + lời giải bài tập của từng chương
```

Lỗi và bẫy thường gặp

- **"Project is incompatible with this version of Expo Go"**: SDK của project khác SDK của Expo Go. Expo Go chỉ hỗ trợ SDK mới nhất. Cách sửa: nâng SDK (`npx expo install expo@^<SDK mới> --fix`).
- **Lỗi đăng nhập trên iPhone**: CLI và Expo Go phải cùng tài khoản. Chạy `npx expo login`, rồi đăng nhập trong Expo Go, bấm **Try Again**.
- **Quét QR không kết nối**: khác mạng Wi-Fi, hoặc firewall. Dùng `--tunnel`.
- **Viết chữ ngoài <Text>**: RN báo lỗi "Text strings must be rendered within a component". Web cho phép chữ trong `<div>`, RN thì không.
- **Dùng npm install cho thư viện native**: nên dùng `npx expo install <pkg>` để lấy đúng phiên bản tương thích với SDK.

Tóm tắt

- React Native vẽ UI native; Expo là framework được docs chính thức khuyên dùng.
- Expo Go giúp chạy app trên iPhone không cần Mac, nhưng chỉ với SDK mới nhất và module có sẵn.
- SDK 57 cần đăng nhập cùng tài khoản ở CLI và Expo Go.
- Luôn chạy `typecheck`, `lint`, `test` trước khi commit.

Bài tập (có lời giải)

Bài 1. Thêm nút **"Đặt lại"** vào `HelloExpo`. Nút đưa bộ đếm về 0 và bị vô hiệu (`disabled`) khi bộ đếm đang bằng 0. Viết test.

▼ Lời giải

File `examples/src/chapters/ch01/exercise.solution.tsx`:

```
export function HelloWithReset() {
  const [count, setCount] = useState(0);
  return (
    <View style={{ padding: 24, gap: 12 }}>
      <Pressable accessibilityRole="button" onPress={() => setCount((c) => c + 1)}>
        <Text>Đã bấm {count} lần</Text>
      </Pressable>
      <Pressable
        accessibilityRole="button"
        disabled={count === 0}
        accessibilityState={{ disabled: count === 0 }}
        onPress={() => setCount(0)}
        style={{ opacity: count === 0 ? 0.4 : 1 }}
      >
        <Text>Đặt lại</Text>
      </Pressable>
    </View>
  );
}
```

Test (`exercise.solution.test.tsx`) dùng matcher `toBeDisabled()` / `toBeEnabled()`:

```
expect(screen.getByRole('button', { name: 'Đặt lại' })).toBeDisabled();
await user.press(screen.getByRole('button', { name: 'Đã bấm 0 lần' }));
expect(screen.getByRole('button', { name: 'Đặt lại' })).toBeEnabled();
```

Vì sao cần cả `disabled` và `accessibilityState`? `disabled` chặn sự kiện bấm;
`accessibilityState` báo cho VoiceOver (trình đọc màn hình của iOS) và cho test biết nút bị vô hiệu.

Bài 2. Chạy `npx expo start`, mở app trên iPhone, rồi lắc máy (shake) để mở **Dev Menu**.
Tìm nút mở React Native DevTools. Ghi lại 3 mục bạn thấy trong menu.

▼ Lời giải

Đây là bài thực hành trên máy thật nên không có đáp án cố định. Bạn thường thấy các mục như **Reload**, **Toggle Element Inspector**, **Open DevTools** (tên mục chính xác có thể khác theo phiên bản — **UNVERIFIED**, chưa xem được trên máy thật). Nếu không lắc được, nhấn `m` trong terminal đang chạy `expo start` để mở Dev Menu trên thiết bị. (Chúng tôi không chạy được bước này trong sandbox — **NOT RUN**.)

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- React Native — Get Started (mã nguồn docs): <https://github.com/facebook/react-native-website/blob/main/docs/getting-started.md>
- Expo — Create a project: <https://github.com/expo/expo/blob/main/docs/pages/get-started/create-a-project.mdx>
- Expo — Start developing (đăng nhập Expo Go): <https://github.com/expo/expo/blob/main/docs/pages/get-started/start-developing.mdx>
- Expo — Development builds introduction: <https://github.com/expo/expo/blob/main/docs/pages/develop/development-builds/introduction.mdx>
- Expo — Upgrading SDK walkthrough: <https://github.com/expo/expo/blob/main/docs/pages/workflow/upgrading-expo-sdk-walkthrough.mdx>
- React Native 0.82 blog (New Architecture only): <https://github.com/facebook/react-native-website/blob/main/website/blog/2025-10-08-react-native-0.82.mdx>
- React Native 0.84 blog (Hermes V1): <https://github.com/facebook/react-native-website/blob/main/website/blog/2026-02-11-react-native-0.84.mdx>
- React Native Testing Library v14 — LLM guidelines (API async): <https://github.com/callstack/react-native-testing-library/blob/main/docs/guides/llm-guidelines.md>

Chương 2 — React Native cho Angular developer

Mục tiêu

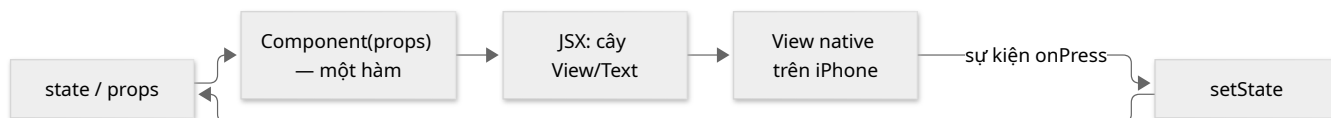
Bạn đã giỏi Angular. Chương này giúp bạn **chuyển kiến thức** sang React Native thật nhanh:

- Đối chiếu từng khái niệm: component, template, service/DI, RxJS/Signals, routing, forms, testing.
- Hiểu điểm khác lớn nhất: React là "**UI = f(state)**", render lại cả hàm component khi state đổi.
- Viết 4 ví dụ nhỏ có test: `@Input/@Output`, Signals, DI, `debounceTime`.

Giải thích đơn giản

Trong Angular, một component là **một class** + **một template HTML**. Angular theo dõi thay đổi (change detection, hoặc Signals) và cập nhật DOM.

Trong React, một component là **một hàm TypeScript** trả về **JSX** (cú pháp giống HTML nằm ngay trong code). Mỗi khi state hoặc props đổi, React **gọi lại hàm đó**, so sánh kết quả mới với cũ, và chỉ cập nhật phần khác (trên RN: cập nhật view native).



Không có `ngOnChanges`, không có `zone.js`, không có module. Chỉ có **hàm**, **hook** (các hàm bắt đầu bằng `use...`) và **props**.

Ví dụ

Bảng đối chiếu chính

Angular	React Native (sách dùng)	Ghi chú
@Component class + template	Hàm component trả về JSX	Không có file .html riêng
<div>, , <p>	<View>, <Text>	Mọi chữ phải nằm trong <Text>
<button (click)>	<Pressable onPress>	Button có sẵn nhưng khó style
<input [(ngModel)]>	<TextInput value onChangeText>	Luôn "controlled"
	<Image source={{ uri }}>	Cần width/height
*ngFor / @for	array.map() hoặc <FlatList>	Danh sách dài → FlatList (Chương 5)
*ngIf / @if	{cond ? <A/> : null}	Toán tử JS thường
[class.active], SCSS	style={[styles.a, active && styles.b]}	StyleSheet.create, không có CSS cascade (Chương 4)
@Input() / input()	props	Chỉ đọc (read-only)
@Output() / output()	callback prop onChange	Hàm được truyền xuống
ngOnInit / ngOnDestroy	useEffect(() => { ...; return cleanup }, [])	Một hook cho cả hai
ngOnChanges	useEffect(..., [prop]) hoặc tính trực tiếp khi render	
signal()	useState() (cục bộ) / store (toàn cục)	Xem ví dụ createSignal
computed()	useMemo() hoặc tính trực tiếp	
effect()	useEffect()	
Service @Injectable + DI	Custom hook + Context, hoặc store (Zustand, Tập 2)	
providers: [{ provide, useClass }]	<Context.Provider value={impl}>	
RxJS Observable	Promise + hook; TanStack Query cho HTTP (Tập 2)	RxJS vẫn dùng được nếu muốn
debounceTime	hook useDebounceValue	
HttpClient	fetch (có sẵn)	Interceptor → hàm wrapper
Angular Router	Expo Router (file-based)	Chương 7
[routerLink]	<Link href>	
ActivatedRoute.paramMap	useLocalSearchParams()	
Route guard canActivate	<Redirect> / Stack.Protected trong layout	Tập 2–3

Angular	React Native (sách dùng)	Ghi chú
Reactive Forms + Validators	useState + hàm validate thuần (Chương 6)	Thư viện: react-hook-form
TestBed + Jasmine/Jest	Jest + jest-expo + React Native Testing Library	
fixture.debugElement.query(By.css)	screen.getByRole(...)	Tìm theo vai trò, như người dùng thấy
Angular CLI ng serve	npx expo start	
ng build	npx expo export / EAS Build	Tập 3

Ví dụ 1 — @Input/@Output → props + callback

Angular:

```
@Component({ selector: 'app-rating', template: `...` })
export class RatingComponent {
  @Input() value = 0;
  @Output() valueChange = new EventEmitter<number>();
}
```

React Native (examples/src/chapters/ch02/RatingStars.tsx):

```
interface RatingStarsProps {
  value: number;
  max?: number;
  onChange: (value: number) => void;
}

export function RatingStars({ value, max = 5, onChange }: RatingStarsProps) {
  return (
    <View style={{ flexDirection: 'row' }}>
      {Array.from({ length: max }, (_, i) => i + 1).map((n) => (
        <Pressable
          key={n}
          accessibilityRole="button"
          accessibilityLabel={` ${n} sao`}
          accessibilityState={{ selected: n <= value }}
          onPress={() => onChange(n)}
        >
          <Text style={{ fontSize: 32 }}>{n <= value ? '★' : '☆'}</Text>
        </Pressable>
      ))}
    </View>
  );
}
```

Component cha giữ state: `const [rating, setRating] = useState(3);` rồi `<RatingStars value={rating} onChange={setRating} />` — đây chính là "banana in a box" [(value)] nhưng viết tường minh.

Ví dụ 2 — Signals → useSyncExternalStore

React không có Signals. Nhưng ý tưởng "một giá trị, ai quan tâm thì đăng ký nghe" rất dễ viết (`ch02/signal.ts`):

```
export function createSignal<T>(initial: T): Signal<T> {
  let value = initial;
  const listeners = new Set<() => void>();
  const set = (next: T) => {
    if (Object.is(next, value)) return; // giống Angular: không đổi thì không báo
    value = next;
    listeners.forEach((l) => l());
  };
  return {
    get: () => value,
    set,
    update: (fn) => set(fn(value)),
    subscribe: (listener) => {
      listeners.add(listener);
      return () => listeners.delete(listener);
    },
  };
}

export function useSignal<T>(signal: Signal<T>): T {
  return useSyncExternalStore(signal.subscribe, signal.get, signal.get);
}
```

`useSyncExternalStore` là hook chính thức của React để đọc một "store bên ngoài". Zustand (Tập 2) cũng dựa trên cùng ý tưởng. Trong app thật bạn sẽ dùng Zustand, không tự viết.

Ví dụ 3 — DI → Context

```
export interface GreetingService { greet: (name: string) => string }

export const friendlyGreeting: GreetingService = { greet: (name) => `Chào ${name}!` };
export const formalGreeting: GreetingService = { greet: (name) => `Kính chào anh/chị ${name}.` };
};

const GreetingContext = createContext<GreetingService>(friendlyGreeting);

export function GreetingProvider({ service, children }: { service: GreetingService; children:
  ReactNode }) {
  const value = useMemo(() => service, [service]);
  return <GreetingContext.Provider value={value}>{children}</GreetingContext.Provider>;
}

// Giống inject(GreetingService)
export function useGreeting(): GreetingService {
  return useContext(GreetingContext);
}
```

- `createContext(default) ~ providedIn: 'root'` với giá trị mặc định.
- `<GreetingProvider service={formalGreeting}> ~ providers: [{ provide: GreetingService, useValue: formalGreeting }]` trong một component con — **hierarchical injector** y như Angular.
- Trong test, bạn bọc component bằng Provider với bản giả (mock) — giống `TestBed.overrideProvider`.

Ví dụ 4 — RxJS `debounceTime` → hook

```
// RxJS: this.search$.pipe(debounceTime(300), distinctUntilChanged())
export function useDebounceValue<T>(value: T, delayMs = 300): T {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delayMs);
    return () => clearTimeout(timer); // chạy khi value đổi hoặc component bị hủy
  }, [value, delayMs]);
  return debounced;
}
```

Hàm `return` trong `useEffect` giống `unsubscribe()` / `takeUntilDestroyed()`.

Kết quả test thật

Chạy ngày 2026-09-28, `npx jest --verbose src/chapters/ch02:`

```
PASS src/chapters/ch02/angular-mapping.test.tsx
  Angular → React Native
    ✓ RatingStars: props đi xuống, callback đi lên (như @Input/@Output)
    ✓ createSignal: set/update/subscribe, bỏ qua giá trị không đổi
    ✓ Context thay implementation giống providers: [{ provide, useClass }]
    ✓ useDebounceValue chỉ cập nhật sau 300ms (debounceTime)
    ✓ Demo: bấm "Thêm vào giỏ" làm badge cập nhật
PASS src/chapters/ch02/exercise.solution.test.ts
  ✓ createComputed tính lại khi nguồn đổi
Test Suites: 2 passed, 2 total
Tests:      6 passed, 6 total
```

Trên Expo Go: tab **Lab** → "Ch.2 — Angular → React Native" (NOT RUN trong sandbox).

Đi sâu

Routing: Angular Router ↔ Expo Router

Angular	Expo Router
<code>const routes: Routes = [{ path: 'task/:id', component: TaskDetail }]</code>	Tạo file <code>src/app/task/[id].tsx</code>
<code><router-outlet></code> + layout component	File <code>_layout.tsx</code> (Stack, Tabs)
<code>router.navigate(['/task', id])</code>	<code>router.push(`/task/\${id}`)</code>
Lazy loading module	Mặc định mỗi route là một file; Metro bundle theo route trên web
Child routes	Thư mục lồng nhau + <code>_layout.tsx</code>

Forms: Reactive Forms ↔ controlled components

Angular có `FormGroup`, `FormControl`, `Validators`, trạng thái `touched/dirty`. React Native không có sẵn thư viện form. Cách đơn giản và dễ test nhất (Chương 6): state bằng `useState`, validate bằng **hàm thuần** `validate(values) → errors`, tự quản `touched`. Khi form lớn, dùng thư viện như `react-hook-form` (không dùng trong sách này để giữ ít phụ thuộc).

Testing: TestBed ↔ React Native Testing Library

Angular	RN (jest-expo + RNTL 14)
<code>TestBed.configureTestingModule({ providers })</code>	<code>render(<X/>, { wrapper: Providers })</code>
<code>fixture.detectChanges()</code>	Không cần; <code>await render()</code> và <code>await user.press()</code> tự xử lý
<code>By.css('.btn')</code>	<code>screen.getByRole('button', { name: 'Lưu' })</code>
<code>fakeAsync + tick(300)</code>	<code>jest.useFakeTimers() + jest.advanceTimersByTime(300)</code>
<code>HttpTestingController</code>	<code>mock fetch</code> (Tập 2)

Chú ý RNTL v14: `render`, `renderHook`, `fireEvent`, `userEvent` đều **async**. Luôn `await`.

Tư duy khác biệt quan trọng

- Immutability (bất biến):** không sửa object/array trong state; tạo bản mới (`[...list, item]`, `{ ...obj, done: true }`). React so sánh bằng tham chiếu.
- Render là hàm thuần:** không gọi API, không `setState` trực tiếp trong thân hàm component. Side effect đặt trong `useEffect` hoặc handler sự kiện.
- Quy tắc của hook:** chỉ gọi hook ở cấp cao nhất của component/hook khác, không trong `if`/vòng lặp. ESLint (`eslint-config-expo`) kiểm tra việc này.

4. **Không có CSS toàn cục:** style gắn trực tiếp vào từng component.

Lỗi và bẫy thường gặp

- **Mutate state:** `tasks.push(t); setTasks(tasks)` → UI không cập nhật vì cùng tham chiếu. Viết `setTasks([...tasks, t])`.
- **Quên dependency trong `useEffect`:** giá trị "cũ" (stale closure). Tin lời cảnh báo của ESLint.
- **Tạo Context value mới mỗi lần render:** mọi consumer render lại. Dùng `useMemo` như ví dụ 3.
- **Mang RxJS vào mọi thứ:** được, nhưng thường không cần. Promise + hook + TanStack Query là đủ.
- **Tìm `ngOnChanges`:** thường bạn chỉ cần **tính trực tiếp** từ props khi render.

Tóm tắt

- Component = hàm; template = JSX; `@Input/@Output` = props/callback.
- Service + DI = hook + Context (hoặc store). Signals $\approx$ `useState/useSyncExternalStore`.
- RxJS operator phổ biến có thể thay bằng hook nhỏ.
- Expo Router = routing theo file. Test theo cách người dùng thấy, và nhớ `await`.

Bài tập (có lời giải)

Bài 1. Viết `createComputed(sources, compute)` — tương đương `computed()` của Angular — dùng `createSignal` ở trên. Khi một nguồn đổi, tính lại; chỉ báo listener khi kết quả đổi.

▼ Lời giải

```
examples/src/chapters/ch02/exercise.solution.ts:
```

```

export function createComputed<T>(sources: Signal<unknown>[], compute: () => T):
ReadonlySignal<T> {
  let value = compute();
  const listeners = new Set<() => void>();
  for (const s of sources) {
    s.subscribe(() => {
      const next = compute();
      if (Object.is(next, value)) return;
      value = next;
      listeners.forEach((l) => l());
    });
  }
  return {
    get: () => value,
    subscribe: (l) => {
      listeners.add(l);
      return () => listeners.delete(l);
    },
  };
}

```

Test: `price = 100`, `qty = 2` → total 200; `qty.set(3)` → total 300, listener được gọi đúng 1 lần. Khác với Angular: Angular tự phát hiện signal nào được đọc (auto-tracking); ở đây ta khai báo nguồn bằng tay. Đó là lý do Angular Signals tiện hơn — và vì sao trong React ta thường chỉ dùng `useMemo(() => price * qty, [price, qty])`.

Bài 2. Viết lại service Angular sau bằng React (hook + Context), rồi nói cách mock nó trong test:

```

@Injectable({ providedIn: 'root' })
export class ClockService { now(): Date { return new Date(); } }

```

▼ Lời giải

```

export interface ClockService { now: () => Date }
const ClockContext = createContext<ClockService>({ now: () => new Date() });
export const useClock = () => useContext(ClockContext);

// Trong test:
const fixed: ClockService = { now: () => new Date(2026, 0, 1) };
await render(<ClockContext.Provider value={fixed}><MyScreen /></ClockContext.Provider>);

```

Mẫu này giống hệt `GreetingService` ở ví dụ 3 (đã được test trong `angular-mapping.test.tsx`).

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- React Native — Core Components and Native Components: <https://github.com/facebook/react-native-website/blob/main/docs/intro-react-native-components.md>
- React Native — Handling Text Input: <https://github.com/facebook/react-native-website/blob/main/docs/handling-text-input.md>
- Expo Router — Core concepts: <https://github.com/expo/expo/blob/main/docs/pages/router/basics/core-concepts.mdx>
- Expo Router — Navigation: <https://github.com/expo/expo/blob/main/docs/pages/router/basics/navigation.mdx>
- React Native Testing Library (hướng dẫn v14 đi kèm gói npm): <https://github.com/callstack/react-native-testing-library>
- Angular Signals (mã nguồn docs angular.dev; trang angular.dev bị chặn trong sandbox): <https://github.com/angular/angular/blob/main/adev/src/content/guide/signals/overview.md>

Chương 3 — Component, JSX, props, state và các core component

Mục tiêu

- Viết component bằng hàm + JSX.
- Dùng các **core component** (component lõi): `View`, `Text`, `Image`, `Pressable`, `ScrollView`.
- Hiểu **props** (dữ liệu vào) và **state** (dữ liệu thay đổi bên trong).
- Dùng `useEffect` cho side effect và dọn dẹp (cleanup).

Giải thích đơn giản

Hãy nghĩ component như một **công thức**: cho vào `props` + `state`, ra **giao diện**.

- **Props** giống tham số hàm. Component con **không được sửa** props.
- **State** là "trí nhớ" của component. Gọi hàm `setX(...)` → React gọi lại component → giao diện mới.
- **Core component** là "thẻ HTML" của React Native. Tài liệu RN liệt kê: `<View>` ~ `<div>`, `<Text>` ~ `<p>`, `<Image>` ~ `<img>`, `<ScrollView>` ~ `<div>` cuộn được, `<TextInput>` ~ `<input>`.

Ví dụ

ProfileCard — props và core component

`examples/src/chapters/ch03/ProfileCard.tsx`:

```

export interface Profile {
  name: string;
  role: string;
  avatar: ImageSourcePropType; // ảnh local: require('...png'); ảnh mạng: { uri: 'https://...' }
}

export function ProfileCard({ profile, onFollow, following }: {
  profile: Profile;
  following: boolean;
  onFollow: () => void;
}) {
  return (
    <View style={styles.card}>
      <Image source={profile.avatar} style={styles.avatar} accessibilityLabel={`Ảnh của ${profile.name}`} />
      <View style={styles.info}>
        <Text style={styles.name}>{profile.name}</Text>
        <Text style={styles.role}>{profile.role}</Text>
      </View>
      <Pressable accessibilityRole="button" onPress={onFollow} style={styles.button}>
        <Text style={styles.buttonText}>{following ? 'Đang theo dõi' : 'Theo dõi'}</Text>
      </Pressable>
    </View>
  );
}

```

Chú ý: `ProfileCard` không tự lưu `following`. Cha quyết định (**lifting state up** — đưa state lên cha). Giống component "dumb/presentational" trong Angular.

Counter — state với `useState`

```

export function Counter({ step = 1 }: { step?: number }) {
  const [count, setCount] = useState(0);
  return (
    <View style={{ flexDirection: 'row', alignItems: 'center', gap: 16 }}>
      <Pressable accessibilityRole="button" accessibilityLabel="Giảm" onPress={() =>
        setCount((c) => c - step)}>
        <Text style={{ fontSize: 28 }}>-</Text>
      </Pressable>
      <Text accessibilityLabel="Giá trị" style={{ fontSize: 28 }}>{count}</Text>
      <Pressable accessibilityRole="button" accessibilityLabel="Tăng" onPress={() =>
        setCount((c) => c + step)}>
        <Text style={{ fontSize: 28 }}>+</Text>
      </Pressable>
    </View>
  );
}

```

`setCount((c) => c + step)` dùng **dạng hàm** (updater): an toàn khi nhiều lần cập nhật liên tiếp.

Clock — useEffect và cleanup

```
export function Clock() {
  const [now, setNow] = useState(() => new Date());
  useEffect(() => {
    const id = setInterval(() => setNow(new Date()), 1000);
    return () => clearInterval(id);
  }, []);
  return <Text accessibilityLabel="Đồng hồ">{formatTime(now)}</Text>;
}
```

- Mảng `[]` = chỉ chạy **một lần** sau lần render đầu (như `onInit`).
- Hàm `return` chạy khi component bị gỡ (như `onDestroy`).

Test và kết quả thật

Test Clock dùng **fake timers** (đồng hồ giả) — giống `fakeAsync` + `tick()`:

```
jest.useFakeTimers();
jest.setSystemTime(new Date(2026, 0, 1, 8, 0, 0));
const clearSpy = jest.spyOn(globalThis, 'clearInterval');
const view = await render(<Clock />);
expect(screen.getByLabelText('Đồng hồ')).toHaveTextContent('08:00:00');
await act(async () => { jest.advanceTimersByTime(2000); });
expect(screen.getByLabelText('Đồng hồ')).toHaveTextContent('08:00:02');
await view.unmount();
expect(clearSpy).toHaveBeenCalled(); // cleanup của useEffect đã chạy
```

Kết quả (2026-09-28, `npx jest --verbose src/chapters/ch03`):

```
PASS src/chapters/ch03/components.test.tsx
  Chương 3 – component, props, state
    ✓ Counter tăng/giảm theo step
    ✓ ProfileCard đổi nhãn nút khi bấm Theo dõi
    ✓ formatTime thêm số 0 phía trước
    ✓ Clock tự cập nhật mỗi giây và dọn interval khi unmount
PASS src/chapters/ch03/exercise.solution.test.tsx
  ✓ LikeButton bật/tắt và đổi số lượt thích
  ✓ Bài 2: Clock dừng khi paused = true và chạy lại khi false
Test Suites: 2 passed, 2 total
Tests:       6 passed, 6 total
```

Xem trên Expo Go: tab **Lab** → "Ch.3" (**NOT RUN** trong sandbox).

Đi sâu

JSX chỉ là lời gọi hàm

`<Text style={s}>Hi</Text>` được biên dịch thành `jsx(Text, { style: s, children: 'Hi' })`. Vì vậy trong `{...}` bạn viết **biểu thức JS**: `{a && <X/>}`, `{list.map(...)}`, nhưng không viết được `if` hay `for` trực tiếp.

Khi nào component render lại?

1. State của nó đổi (`setX` với giá trị khác — so sánh bằng `Object.is`).
2. Cha render lại (kể cả khi props giống cũ!) — trừ khi bọc bằng `memo`.
3. Context mà nó dùng đổi.

Đây giống `ChangeDetectionStrategy.Default`. `memo(Component)` giống `OnPush`. Tập 3 (Performance) nói kỹ hơn, kèm React Compiler.

Ảnh: local và từ mạng

- Local: `source={require('./avatar.png')}` — Metro đóng gói ảnh, tự biết kích thước.
- Mạng: `source={{ uri: 'https://...' }}` — **phải** đặt `width/height` trong style, nếu không ảnh có kích thước 0.

Pressable và accessibility

`Pressable` có `onPress`, `onLongPress`, `hitSlop` (mở rộng vùng chạm), và style dạng hàm `style={({ pressed }) => ...}` để làm hiệu ứng khi nhấn. Luôn thêm `accessibilityRole="button"` và nhãn (`accessibilityLabel` hoặc chữ bên trong) để VoiceOver đọc được — và để test tìm được bằng `getByRole('button', { name })`.

Lỗi và bẫy thường gặp

- **Chữ nằm ngoài** `<Text>` → lỗi đỏ khi chạy.
- **Sửa state trực tiếp**: `count++` không làm gì. Luôn dùng `setCount`.
- **Gọi** `setState` **trong thân component** → vòng lặp render vô hạn.
- **Quên cleanup** `interval/listener` → rò rỉ bộ nhớ, cảnh báo "state update on unmounted component".
- **Ảnh mạng không hiện**: quên `width/height`.
- `onPress={doIt()}` (gọi ngay) thay vì `onPress={doIt}` hoặc `onPress={() => doIt(x)}`.

Tóm tắt

- Component = hàm(props) → JSX. Props đi xuống, sự kiện đi lên qua callback.

- `useState` cho dữ liệu thay đổi; dùng updater `setX(prev => ...)`.
- `useEffect` cho side effect; `return` để dọn dẹp.
- Core component: `View`, `Text`, `Image`, `Pressable`, `ScrollView`, `TextInput`.

Bài tập (có lời giải)

Bài 1. Viết `LikeButton` nhận `initialLikes`. Bấm lần 1: ❤️ và +1; bấm lần 2: 💔 và trở lại. Nhấn accessibility đổi giữa "Thích"/"Bỏ thích".

▼ Lời giải

`examples/src/chapters/ch03/exercise.solution.tsx`:

```
export function LikeButton({ initialLikes = 0 }: { initialLikes?: number }) {
  const [liked, setLiked] = useState(false);
  const likes = initialLikes + (liked ? 1 : 0); // "derived state": tính từ state, không
  lưu thêm
  return (
    <Pressable
      accessibilityRole="button"
      accessibilityState={{ selected: liked }}
      accessibilityLabel={liked ? 'Bỏ thích' : 'Thích'}
      onPress={() => setLiked((v) => !v)}
    >
      <Text>{liked ? '❤️' : '💔'} {likes}</Text>
    </Pressable>
  );
}
```

Điểm chính: **không** tạo state thứ hai `likes`. Giá trị suy ra được thì tính khi render (giống `computed()`), tránh hai state lệch nhau.

Bài 2. Đồng hồ `Clock` chạy mãi. Hãy thêm prop `paused: boolean`; khi `paused = true` thì dừng `interval`. Gợi ý: dependency của `useEffect`.

▼ Lời giải

```
useEffect(() => {
  if (paused) return; // không tạo interval
  const id = setInterval(() => setNow(new Date()), 1000);
  return () => clearInterval(id); // chạy khi paused đổi hoặc unmount
}, [paused]);
```

Khi `paused` đổi từ `false` → `true`, React chạy cleanup của effect cũ (xóa `interval`), rồi chạy effect mới (thoát sớm). Đây là mẫu "đăng ký lại khi input đổi", giống `switchMap` trong RxJS.

Lời giải đã có trong `ch03/Clock.tsx` (prop `paused`) và được test trong `ch03/exercise.solution.test.tsx` ("Bài 2: Clock dừng khi `paused = true...`"): dùng `view.render(<Clock paused={false} />)` để đổi prop.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- Core Components and Native Components: <https://github.com/facebook/react-native-website/blob/main/docs/intro-react-native-components.md>
- React Fundamentals (props, state): <https://github.com/facebook/react-native-website/blob/main/docs/intro-react.md>
- Props: <https://github.com/facebook/react-native-website/blob/main/docs/props.md>
- State: <https://github.com/facebook/react-native-website/blob/main/docs/state.md>
- Images: <https://github.com/facebook/react-native-website/blob/main/docs/images.md>
- Pressable: <https://github.com/facebook/react-native-website/blob/main/docs/pressable.md>

Chương 4 — Styling và Flexbox

Mục tiêu

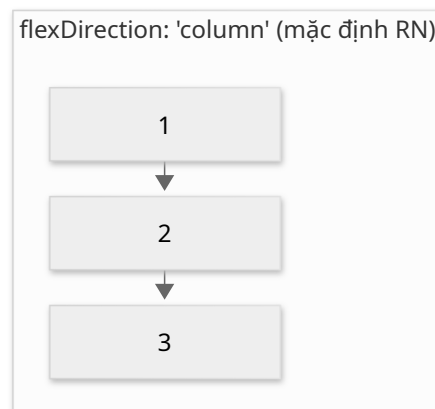
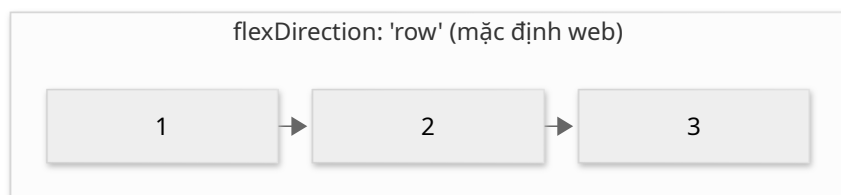
- Viết style bằng `StyleSheet.create` và mảng style.
- Nắm **Flexbox** trong RN và các khác biệt so với CSS web.
- Làm layout thường gặp: hàng/cột, căn giữa, lưới 2 cột, nút nổi (floating button).
- Biết cách xử lý kích thước màn hình, dark mode, và code riêng cho từng nền tảng.

Giải thích đơn giản

React Native **không có CSS**. Style là **object JavaScript**. Tên thuộc tính viết kiểu camelCase: `backgroundColor` thay cho `background-color`. Số **không có đơn vị**: 16 là 16 "điểm" (density-independent pixels — điểm độc lập mật độ), iPhone tự nhân theo độ nét màn hình.

Mọi `View` đều là một **flex container**. Chỉ cần nhớ 3 thuộc tính chính:

- `flexDirection`: trục chính đi dọc (`column`, **mặc định**) hay ngang (`row`).
- `justifyContent`: xếp con trên **trục chính**.
- `alignItems`: xếp con trên **trục phụ** (vuông góc).



Ví dụ

Style cơ bản và mảng style

```
const styles = StyleSheet.create({
  base: { paddingVertical: 12, paddingHorizontal: 16, borderRadius: 10, alignItems: 'center' },
  primary: { backgroundColor: '#2563eb' },
  pressed: { opacity: 0.7 },
  disabled: { opacity: 0.4 },
});

<Pressable style={({ pressed }) => [styles.base, styles.primary, pressed && styles.pressed,
disabled && styles.disabled]} />
```

Trong mảng, style **sau thắng style trước** (giống thứ tự CSS). Giá trị `false/null` bị bỏ qua, nên viết `cond && styles.x` rất tiện — giống `[class.x]="cond"`. Đoạn trên lấy từ

`examples/src/components/AppButton.tsx`.

Flexbox playground

`examples/src/chapters/ch04/FlexPlayground.tsx` cho bạn bấm để đổi `flexDirection`, `justifyContent`, `alignItems` và thấy ngay kết quả trên iPhone (tab **Lab** → "Ch.4"). Logic đổi lựa chọn là một hàm thuần:

```
export const DIRECTIONS = ['column', 'row', 'column-reverse', 'row-reverse'] as const;

export function nextOption<T>(options: readonly T[], current: T): T {
  const i = options.indexOf(current);
  return options[(i + 1) % options.length];
}
```

Test kiểm tra style thật trên view bằng matcher `toHaveStyle`:

```
await render(<FlexPlayground />);
expect(screen.getByTestId('flex-box')).toHaveStyle({ flexDirection: 'column' });
await user.press(screen.getByRole('button', { name: 'flexDirection: column' }));
expect(screen.getByTestId('flex-box')).toHaveStyle({ flexDirection: 'row' });
```

Kết quả (2026-09-28):

```
PASS src/chapters/ch04/flex.test.tsx
  Chương 4 – Flexbox
    ✓ nextOption quay vòng về đầu danh sách
    ✓ mặc định là column; bấm nút đổi sang row
PASS src/chapters/ch04/exercise.solution.test.tsx
  ✓ TwoColumnGrid dùng row + wrap, mỗi ô rộng 48%
  ✓ Bài 2: columnsFor và ResponsiveGrid trên màn hình iPad
Test Suites: 2 passed, 2 total
Tests:      4 passed, 4 total
```

Layout thường gặp

Muốn	Style
Chiếm hết màn hình	<code>{ flex: 1 }</code>
Căn giữa cả hai chiều	<code>{ flex: 1, justifyContent: 'center', alignItems: 'center' }</code>
Một hàng: icon + chữ co giãn + nút	cha <code>{ flexDirection: 'row', alignItems: 'center', gap: 12 }</code> , chữ <code>{ flex: 1 }</code>
Khoảng cách giữa các con	<code>gap, rowGap, columnGap</code>
Nút nổi góc phải dưới	<code>{ position: 'absolute', right: 24, bottom: 24 }</code>
Lưới 2 cột	cha <code>{ flexDirection: 'row', flexWrap: 'wrap', justifyContent: 'space-between' }</code> , con <code>{ width: '48%' }</code>

Ví dụ thật: hàng công việc trong `src/features/tasks/TaskItem.tsx` dùng `flexDirection: 'row' + gap` + phần chữ `flex: 1`; nút "+" ở `src/app/(tabs)/index.tsx` dùng `position: 'absolute'`.

Đi sâu

Khác biệt với CSS web (theo tài liệu RN)

Tài liệu Flexbox của RN nói rõ: `flexDirection` mặc định `column` (web: `row`), `alignContent` mặc định `flex-start` (web: `stretch`), `flexShrink` mặc định `0` (web: `1`), và `flex` chỉ nhận **một số**.

Ngoài ra:

- **Không kế thừa style** (no cascade) — trừ trong `<Text>` lồng nhau (Text con kế thừa font/màu từ Text cha).
- **Không có selector**, không `:hover`, không media query. Dùng JS: `useWindowDimensions()`.
- % được hỗ trợ cho width/height/margin... nhưng cha phải có kích thước xác định.
- `StyleSheet.hairlineWidth`: đường kẻ mảnh nhất mà màn hình vẽ được — đẹp cho dòng phân cách.

Responsive: useWindowDimensions

```
const { width } = useWindowDimensions();
const columns = width >= 768 ? 3 : 2; // iPad xoay ngang → 3 cột
```

Hook này tự cập nhật khi xoay màn hình. Tránh `Dimensions.get()` một lần ở đầu file (không cập nhật).

Dark mode

`useColorScheme()` trả về `'light' | 'dark'`. App ví dụ đặt `"userInterfaceStyle": "automatic"` trong `app.json` để app theo cài đặt của iPhone. Mẫu đơn giản:

```
const scheme = useColorScheme();
const bg = scheme === 'dark' ? '#111827' : 'ffffff';
```

Code riêng cho iOS/Android

- `Platform.OS === 'ios'` hoặc `Platform.select({ ios: 12, android: 8 })`.
- File riêng: `Button.ios.tsx` và `Button.android.tsx` — Metro tự chọn.
- Ví dụ trong sách: `KeyboardAvoidingView behavior={Platform.OS === 'ios' ? 'padding' : undefined}` (Chương 6).

Safe area (tai thỏ, Dynamic Island)

Màn hình dùng Expo Router (Stack/Tabs) đã tự tránh phần header và thanh tab. Khi bạn tự làm màn hình full-screen, dùng `SafeAreaView` hoặc hook `useSafeAreaInsets()` của `react-native-safe-area-context` (đã có trong dự án).

Lỗi và bẫy thường gặp

- Quên `flex: 1` ở màn hình gốc → nội dung co lại, `FlatList` không cuộn được.
- Nghĩ mặc định là `row` như web → mọi thứ xếp dọc.
- Chữ bị tràn trong hàng: cho phần chữ `flex: 1` và `numberOfLines={1}`.
- `width: '50%'` hai ô + `gap` → tràn dòng. Dùng `48%` + `space-between`, hoặc tính bằng `useWindowDimensions`.
- `StyleSheet.absoluteFillObject`: đã bị xóa ở RN 0.85. Dùng `StyleSheet.absoluteFill`.
- `Style inline object mới mỗi render` (`style={{...}}`) không sai, nhưng trong danh sách dài nên dùng `StyleSheet.create`.

Tóm tắt

- Style = object JS, camelCase, số không đơn vị. Mảng style: sau thẳng trước.

- Flexbox RN: mặc định `column`, `flexShrink: 0`, `alignContent: 'flex-start'`.
- `gap`, `flexWrap`, `position: 'absolute'`, % giải quyết hầu hết layout.
- Responsive bằng `useWindowDimensions`, dark mode bằng `useColorScheme`.

Bài tập (có lời giải)

Bài 1. Làm component `TwoColumnGrid` nhận `items: string[]` và hiển thị lưới 2 cột ô vuông, khoảng cách giữa các hàng 12. Viết test kiểm tra style.

▼ Lời giải

`examples/src/chapters/ch04/exercise.solution.tsx`:

```
export function TwoColumnGrid({ items }: { items: string[] }) {
  return (
    <View testID="grid" style={styles.grid}>
      {items.map((label) => (
        <View key={label} testID="cell" style={styles.cell}>
          <Text>{label}</Text>
        </View>
      ))}
    </View>
  );
}

const styles = StyleSheet.create({
  grid: { flexDirection: 'row', flexWrap: 'wrap', justifyContent: 'space-between', rowGap: 12, padding: 12 },
  cell: { width: '48%', aspectRatio: 1, borderRadius: 12, backgroundColor: '#dbeafe', alignItems: 'center', justifyContent: 'center' },
});
```

`aspectRatio: 1` làm ô vuông mà không cần biết chiều rộng thật. Test:

```
expect(screen.getByTestId('grid')).toHaveStyle({ flexDirection: 'row', flexWrap: 'wrap' }).
```

Bài 2. Sửa `TwoColumnGrid` để có 3 cột khi màn hình rộng ≥ 768 (iPad), 2 cột khi hẹp hơn.

▼ Lời giải

```

export function columnsFor(width: number): 2 | 3 {
  return width >= 768 ? 3 : 2;
}

// Tách phần "vẽ" (nhận width qua props) khỏi phần "đọc môi trường" (hook) → dễ test.
export function ResponsiveGrid({ items }: { items: string[] }) {
  const { width } = useWindowDimensions(); // tự cập nhật khi xoay màn hình
  return <ResponsiveGridView items={items} width={width} />;
}

export function ResponsiveGridView({ items, width }: { items: string[]; width: number }) {
  const cellWidth = columnsFor(width) === 3 ? '31%' : '48%';
  return (
    <View style={styles.grid}>
      {items.map((label) => (
        <View key={label} testID="rcell" style={[styles.cell, { width: cellWidth }]}>
          <Text>{label}</Text>
        </View>
      ))}
    </View>
  );
}

```

Test: `await render(<ResponsiveGridView items={['A','B','C']} width={1024} />)` rồi kiểm tra ô có `width: '31%'`.

Bài học từ lúc viết test: lần đầu chúng tôi thử `import * as RN from 'react-native' + jest.spyOn(RN, 'useWindowDimensions')` nhưng **không có tác dụng** (component vẫn nhận giá trị thật), vì `import` đã được Babel gắn cố định. Cách sạch hơn là tách component "vẽ" nhận `width` qua props — giống việc inject một service trong Angular thay vì gọi `window.innerWidth` trực tiếp.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- Style: <https://github.com/facebook/react-native-website/blob/main/docs/style.md>
- Height and Width: <https://github.com/facebook/react-native-website/blob/main/docs/height-and-width.md>
- Layout with Flexbox: <https://github.com/facebook/react-native-website/blob/main/docs/flexbox.md>
- useWindowDimensions: <https://github.com/facebook/react-native-website/blob/main/docs/usewindowdimensions.md>
- useColorScheme: <https://github.com/facebook/react-native-website/blob/main/docs/usecolorscheme.md>

- Platform-specific code: <https://github.com/facebook/react-native-website/blob/main/docs/platform-specific-code.md>
- React Native 0.85 blog (xóa `absoluteFillObject`): <https://github.com/facebook/react-native-website/blob/main/website/blog/2026-04-07-react-native-0.85.mdx>

Chương 5 — Danh sách: FlatList và SectionList

Mục tiêu

- Biết khi nào dùng `ScrollView`, `FlatList`, `SectionList`.
- Hiển thị danh sách có tìm kiếm, danh sách có nhóm, trạng thái rỗng, kéo để làm mới.
- Xử lý **tiếng Việt có dấu** khi tìm kiếm và sắp xếp.
- Viết test cho danh sách.

Giải thích đơn giản

- `ScrollView` render **tất cả** con cùng lúc. Tốt cho màn hình ngắ (form, trang chi tiết).
- `FlatList` chỉ render các dòng **đang thấy** trên màn hình (virtualization — ảo hóa), giống `cdk-virtual-scroll-viewport` của Angular CDK. Dùng cho danh sách dài.
- `SectionList` là `FlatList` có **tiêu đề nhóm** (A, B, C... như danh bạ iPhone).

`FlatList` cần 3 thứ: `data` (mảng), `renderItem` (vẽ một dòng), `keyExtractor` (khóa duy nhất — giống `track item.id` trong `@for`).

Ví dụ

Dữ liệu và hàm thuần

`examples/src/chapters/ch05/contacts.ts`:

```
// Bỏ dấu tiếng Việt để tìm kiếm và nhóm: "Ánh" → "anh", "Đặng" → "dang".
export function removeDiacritics(s: string): string {
  return s
    .normalize('NFD')
    .replace(/[\u0300-\u036f]/g, '')
    .replace(/đ/g, 'd')
    .replace(/Đ/g, 'D');
}

export function searchContacts(list: Contact[], query: string): Contact[] {
  const q = removeDiacritics(query.trim().toLowerCase());
  if (!q) return list;
  return list.filter((c) => removeDiacritics(c.name.toLowerCase()).includes(q) ||
    c.phone.replace(/\s/g, '').includes(q));
}
```

`normalize('NFD')` tách "á" thành "a" + dấu sắc; regex xóa phần dấu. Chữ **đ/Đ** không tách được theo cách này nên phải thay riêng — một bẫy rất hay gặp với tiếng Việt.

FlatList có tìm kiếm

```
export function ContactList({ contacts = CONTACTS }: { contacts?: Contact[] }) {
  const [query, setQuery] = useState('');
  const data = useMemo(() => searchContacts(contacts, query), [contacts, query]);
  return (
    <View style={{ flex: 1 }}>
      <TextInput accessibilityLabel="Tìm liên hệ" placeholder="Tìm tên hoặc số" value={query}
        onChangeText={setQuery} style={styles.search} />
      <FlatList
        data={data}
        keyExtractor={(c) => c.id}
        renderItem={({ item }) => <Row contact={item} />}
        ItemSeparatorComponent={() => <View style={styles.sep} />}
        ListEmptyComponent={() => <Text style={styles.empty}>Không tìm thấy</Text>}
      />
    </View>
  );
}
```

SectionList có nhóm chữ cái

```
export function groupByLetter(list: Contact[]): ContactSection[] {
  const map = new Map<string, Contact[]>();
  for (const c of list) {
    const letter = removeDiacritics(c.name.charAt(0)).toUpperCase();
    map.set(letter, [...(map.get(letter) ?? []), c]);
  }
  return [...map.entries()]
    .sort(([a], [b]) => a.localeCompare(b))
    .map(([title, data]) => ({ title, data: [...data].sort((x, y) =>
      x.name.localeCompare(y.name, 'vi')) }));
}
```

```
<SectionList
  sections={sections}
  keyExtractor={(c) => c.id}
  renderItem={({ item }) => <Row contact={item} />}
  renderSectionHeader={({ section }) => <Text accessibilityRole="header" style=
    {styles.header}>{section.title}</Text>}
  stickySectionHeadersEnabled
/>
```

Kết quả test thật (2026-09-28)

```
PASS src/chapters/ch05/lists.test.tsx
Chương 5 – danh sách
  ✓ removeDiacritics xử lý cả chữ đ/Đ
  ✓ searchContacts tìm không dấu và theo số điện thoại
  ✓ groupByLetter gom "An" và "Ảnh" vào nhóm A
  ✓ ContactList lọc khi gõ
  ✓ Bài tập 2: phân biệt "Danh bạ trống" và "Không tìm thấy"
  ✓ ContactSections hiện tiêu đề nhóm
PASS src/chapters/ch05/exercise.solution.test.tsx
  ✓ sortContacts theo tiếng Việt
  ✓ bấm nút đổi thứ tự, kéo để làm mới gọi load()
Test Suites: 2 passed, 2 total
Tests:      8 passed, 8 total
```

Trên Expo Go: tab **Lab** → "Ch.5 — FlatList + tìm kiếm" và "Ch.5 — SectionList" (**NOT RUN**).

Đi sâu

Hiệu năng FlatList

Tài liệu RN có trang "Optimizing FlatList Configuration". Các ý chính:

- `keyExtractor` trả về id ổn định (không dùng index nếu danh sách thay đổi thứ tự).
- Component dòng bọc `memo` (xem `TaskItem` ở Chương 8) và callback ổn định (`useCallback`), để dòng không render lại khi không đổi.
- `initialNumToRender`, `windowSize`: đánh đổi giữa bộ nhớ và khoảng trắng khi cuộn nhanh.
- `getItemLayout`: nếu mọi dòng cùng chiều cao, cho FlatList biết trước để khởi đo.
- FlatList là `PureComponent`: nếu dòng phụ thuộc state bên ngoài `data`, truyền `extraData={selectedId}` để nó biết cần render lại.

Tập 3 so sánh với `@shopify/flash-list` (có trong Expo Go SDK 57).

Sắp xếp tiếng Việt: `localeCompare(..., 'vi')`

Bảng chữ cái tiếng Việt: A Ă Â B C D Đ E Ê ... Với `localeCompare(b, 'vi')`, "Đặng" đứng **sau** "Dũng". Test ở bài tập chứng minh điều này (sắp Z → A thì "Đặng Minh" đứng đầu). Nếu chỉ dùng `a < b`, thứ tự sẽ theo mã Unicode và sai với người Việt.

Lưu ý: kết quả `localeCompare` phụ thuộc dữ liệu ICU của môi trường chạy. Test trong sách chạy trên Node 22 (có full ICU). Trên iPhone, Hermes dùng dữ liệu locale của hệ điều hành — chúng tôi **chưa kiểm tra** trên máy thật (**UNVERIFIED**).

Pull-to-refresh

`FlatList` có sẵn `refreshing` + `onRefresh`: kéo xuống ở đầu danh sách → hiện vòng xoay → gọi `onRefresh`. Không cần thư viện.

Lỗi và bẫy thường gặp

- `FlatList` trong `ScrollView` cùng chiều → cảnh báo "VirtualizedLists should never be nested" và mất ảo hóa. Dùng `ListHeaderComponent` / `ListFooterComponent` thay vì bọc `ScrollView`.
- Thiếu `keyExtractor` / `key` trùng → dòng nhảy lung tung khi lọc.
- Tìm kiếm không ra "Đặt" khi gõ "dang": quên thay đ.
- `FlatList` không cuộn: cha không có `flex: 1` (không có chiều cao xác định).
- Dòng không cập nhật khi chọn: quên `extraData`.

Tóm tắt

- Ngắn → `ScrollView`; dài → `FlatList`; có nhóm → `SectionList`.
- Luôn có `keyExtractor`, `ListEmptyComponent`, và `flex: 1` ở cha.
- Tiếng Việt: `normalize('NFD')` + thay đ/đ; sắp xếp bằng `localeCompare(..., 'vi')`.

Bài tập (có lời giải)

Bài 1. Thêm nút đổi thứ tự $A \rightarrow Z / Z \rightarrow A$ và **kéo để làm mới** (gọi một hàm `load()` trả Promise). Viết test cho cả hai.

▼ Lời giải

`examples/src/chapters/ch05/exercise.solution.tsx`:

```

export function sortContacts(list: Contact[], dir: SortDir): Contact[] {
  const sorted = [...list].sort((a, b) => a.name.localeCompare(b.name, 'vi'));
  return dir === 'asc' ? sorted : sorted.reverse();
}

export function SortableContacts({ load = async () => CONTACTS }: { load?: () =>
Promise<Contact[]> }) {
  const [dir, setDir] = useState<SortDir>('asc');
  const [items, setItems] = useState(CONTACTS);
  const [refreshing, setRefreshing] = useState(false);
  const data = useMemo(() => sortContacts(items, dir), [items, dir]);

  const onRefresh = useCallback(async () => {
    setRefreshing(true);
    setItems(await load());
    setRefreshing(false);
  }, [load]);

  return (
    <View style={{ flex: 1 }}>
      <Pressable accessibilityRole="button" onPress={() => setDir((d) => (d === 'asc' ?
'desc' : 'asc'))}>
        <Text style={{ padding: 12 }}>{dir === 'asc' ? 'Sắp xếp: A-Z' : 'Sắp xếp: Z-A'}
      </Text>
      </Pressable>
      <FlatList testID="sortable-list" data={data} keyExtractor={(c) => c.id}
        renderItem={({ item }) => <Text style={{ padding: 12 }}>{item.name}</Text>}
        refreshing={refreshing} onRefresh={onRefresh} />
    </View>
  );
}

```

Trong test, kéo để làm mới được giả lập bằng `await fireEvent(list, 'refresh')`. Chú ý `[...list].sort()` — `sort()` sửa mảng tại chỗ, nên phải sao chép trước (immutability).

Bài 2. Trong `ContactList`, khi danh sách rỗng vì **chưa có dữ liệu** (khác với "không tìm thấy"), hiện "Danh bạ trống". Chỉ sửa `ListEmptyComponent`.

▼ Lời giải

```

ListEmptyComponent={
  <Text style={styles.empty}>{contacts.length === 0 ? 'Danh bạ trống' : 'Không tìm thấy'}
</Text>
}

```

Lời giải này đã được áp dụng trong `ContactList.tsx`, và test nằm trong `lists.test.tsx` ("Bài tập 2: phân biệt ..."): `await render(<ContactList contacts={[]} />)` rồi `expect(screen.getByText('Danh bạ trống')).toBeOnTheScreen()`.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- Using List Views: <https://github.com/facebook/react-native-website/blob/main/docs/using-a-listview.md>
- FlatList: <https://github.com/facebook/react-native-website/blob/main/docs/flatlist.md>
- SectionList: <https://github.com/facebook/react-native-website/blob/main/docs/sectionlist.md>
- ScrollView: <https://github.com/facebook/react-native-website/blob/main/docs/scrollview.md>
- Optimizing FlatList Configuration: <https://github.com/facebook/react-native-website/blob/main/docs/optimizing-flatlist-configuration.md>

Chương 6 — Form và nhập liệu

Mục tiêu

- Dùng `TextInput`, `Switch` theo kiểu **controlled** (giá trị nằm trong state).
- Validate bằng **hàm thuần**, hiển thị lỗi đúng lúc (touched / submitted).
- Xử lý bàn phím iPhone: loại bàn phím, nút "Next", `KeyboardAvoidingView`.
- Test form bằng `userEvent.type` và `userEvent.press`.

Giải thích đơn giản

Trong Angular, bạn có `FormGroup` giữ giá trị, trạng thái và lỗi. React Native không có sẵn thứ đó. Ta tự ghép 3 mảnh nhỏ:

1. **State giá trị:** `const [values, setValues] = useState(EMPTY)`.
2. **Hàm validate thuần:** `validateSignUp(values) → { email?: string, ... }` (giống `Validators`, nhưng chỉ là một hàm — test cực dễ).
3. **Khi nào hiện lỗi:** sau khi rời ô (`onBlur` → `touched`) hoặc sau khi bấm Gửi (`submitted`).

Ví dụ

Hàm validate

`examples/src/chapters/ch06/validation.ts`:

```
const EMAIL_RE = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

export function validateSignUp(v: SignUpValues): SignUpErrors {
  const e: SignUpErrors = {};
  if (!v.email.trim()) e.email = 'Vui lòng nhập email';
  else if (!EMAIL_RE.test(v.email.trim())) e.email = 'Email không hợp lệ';
  if (v.password.length < 8) e.password = 'Mật khẩu cần ít nhất 8 ký tự';
  else if (!/\d/.test(v.password)) e.password = 'Mật khẩu cần ít nhất 1 chữ số';
  if (v.confirm !== v.password) e.confirm = 'Mật khẩu nhập lại không khớp';
  if (!v.acceptTerms) e.acceptTerms = 'Bạn cần đồng ý điều khoản';
  return e;
}

export const isValid = (e: SignUpErrors) => Object.keys(e).length === 0;
```

Form đăng ký

Trích `examples/src/chapters/ch06/SignUpForm.tsx`:

```

const errors: SignUpErrors = validateSignUp(values); // tính lại mỗi lần render: đơn giản, đủ
nhanh
const show = (k: keyof SignUpValues) => (submitted || touched[k]) && errors[k];

<TextInput
  accessibilityLabel="Email"
  placeholder="email@vidu.com"
  value={values.email}
  onChangeText={({t}) => set('email', t)}
  onBlur={blur('email')}
  autoCapitalize="none"
  autoComplete="email"
  keyboardType="email-address"
  returnKeyType="next"
  onSubmitEditing={() => passwordRef.current?.focus()}
  style={styles.input}
/>
{show('email') ? <Text style={styles.error}>{errors.email}</Text> : null}

<TextInput
  ref={passwordRef}
  accessibilityLabel="Mật khẩu"
  secureTextEntry
  textContentType="newPassword"
  returnKeyType="next"
  onSubmitEditing={() => confirmRef.current?.focus()}
  /* ... */
/>

```

- `keyboardType="email-address"` : iPhone hiện bàn phím có @.
- `autoCapitalize="none"` : không tự viết hoa chữ đầu email.
- `secureTextEntry` : ẩn mật khẩu. `textContentType="newPassword"` : iOS gợi ý mật khẩu mạnh.
- `returnKeyType="next"` + `onSubmitEditing` + `ref.focus()` : bấm "Next" nhảy sang ô tiếp theo.

Toàn bộ form nằm trong:

```

<KeyboardAvoidingView style={{ flex: 1 }} behavior={Platform.OS === 'ios' ? 'padding' :
undefined}>
  <ScrollView contentContainerStyle={styles.form} keyboardShouldPersistTaps="handled">
    {/* ... */}
  </ScrollView>
</KeyboardAvoidingView>

```

để bàn phím không che ô nhập, và bấm nút khi bàn phím đang mở vẫn ăn ngay lần đầu.

Test

```
it('gửi form hợp lệ', async () => {
  const onSubmit = jest.fn();
  const user = userEvent.setup();
  await render(<SignUpForm onSubmit={onSubmit} />);
  await user.type(screen.getByLabelText('Email'), 'nobin@vidu.vn');
  await user.type(screen.getByLabelText('Mật khẩu'), 'matkhau1');
  await user.type(screen.getByLabelText('Nhập lại mật khẩu'), 'matkhau1');
  await fireEvent(screen.getByLabelText('Đồng ý điều khoản'), 'valueChange', true);
  await user.press(screen.getByRole('button', { name: 'Đăng ký' }));
  expect(onSubmit).toHaveBeenCalledWith({ email: 'nobin@vidu.vn', password: 'matkhau1',
  confirm: 'matkhau1', acceptTerms: true });
});
```

`userEvent.type` gõ từng ký tự (focus → keyPress → changeText → blur) như người thật.
`userEvent.v14` chưa có hàm riêng cho `Switch` (chỉ có `press`, `longPress`, `type`, `clear`, `paste`, `scrollTo...`), nên dùng `fireEvent(e1, 'valueChange', true)`.

Kết quả (2026-09-28):

```
PASS src/chapters/ch06/forms.test.tsx
  Chương 6 – form
    ✓ validateSignUp trả lỗi đúng từng trường
    ✓ không hiện lỗi trước khi người dùng chạm vào form
    ✓ gửi form hợp lệ
    ✓ bấm Đăng ký khi trống thì hiện tất cả lỗi
PASS src/chapters/ch06/exercise.solution.test.ts
  ✓ validatePhone("0901 234 567") → undefined
  ✓ validatePhone("+84 901 234 567") → undefined
  ✓ validatePhone("090.123.4567") → undefined
  ✓ validatePhone("12345") → "Số điện thoại cần 10 chữ số và bắt đầu bằng 0"
  ✓ validatePhone("") → "Vui lòng nhập số điện thoại"
  ✓ normalizePhone đổi +84 thành 0
Test Suites: 2 passed, 2 total
Tests:      10 passed, 10 total
```

Form thứ hai trong sách: `src/features/tasks/TaskForm.tsx` (app mẫu, Chương 8) dùng cùng mẫu, thêm nhóm radio chọn mức ưu tiên với `accessibilityRole="radio"`.

Đi sâu

So sánh với Reactive Forms

Reactive Forms	Mẫu trong sách
<code>new FormGroup({ email: new FormControl('') })</code>	<code>useState<SignUpValues>(EMPTY)</code>
<code>Validators.required, Validators.email</code>	<code>validateSignUp()</code> thuần
<code>control.touched</code>	<code>touched[k]</code> cập nhật ở <code>onBlur</code>
<code>form.invalid</code>	<code>!isValid(errors)</code>
<code>form.patchValue(x)</code>	<code>setValues((prev) => ({ ...prev, ...x }))</code>
<code>valueChanges</code>	chính là render lại; dùng <code>useEffect</code> nếu cần side effect

Khi form lớn (nhiều bước, mảng động), cân nhắc thư viện `react-hook-form` (hiệu năng tốt vì dùng uncontrolled input) kết hợp `zod` cho schema. Sách không dùng để giữ ít phụ thuộc.

Tính lỗi mỗi lần render có chậm không?

Không đáng kể với form nhỏ: vài phép so sánh chuỗi. Lợi ích: lỗi **luôn khớp** với giá trị, không có state lỗi riêng để quên cập nhật. Nếu validate nặng (ví dụ gọi server), dùng `debounce` (`useDebounceValue` ở Chương 2) và làm trong `useEffect`.

Bàn phím

- `keyboardType: default, number-pad, decimal-pad, email-address, phone-pad ...`
- `KeyboardAvoidingView` trên iOS thường dùng `behavior="padding"`.
- `Keyboard.dismiss()` để ẩn bàn phím chủ động.
- `keyboardShouldPersistTaps="handled"` trên `ScrollView` để bấm nút không bị "nuốt" lần chạm đầu.

Lỗi và bẫy thường gặp

- **Hiện lỗi ngay khi mở form** → khó chịu. Chỉ hiện sau blur hoặc submit.
- **Quên** `autoCapitalize="none"` cho email/username → iPhone viết hoa chữ đầu.
- **Bàn phím che nút Gửi**: thiếu `KeyboardAvoidingView` hoặc `ScrollView`.
- **Validate trên giá trị cũ**: đọc `values` ngay sau `setValues(...)` — state chỉ đổi ở lần render sau. Tính trên object `next` bạn vừa tạo (xem `TaskForm.setField`).
- **Test gõ vào ô không tìm thấy**: thêm `accessibilityLabel` cho `TextInput`.

Tóm tắt

- Form = state giá trị + hàm validate thuần + quy tắc hiển thị lỗi.

- Cấu hình bàn phím đúng làm app "cảm giác native" hơn.
- Test bằng `userEvent.type/press`; Switch dùng `fireEvent(..., 'valueChange', ...)`.

Bài tập (có lời giải)

Bài 1. Viết `validatePhone(input)` cho số di động Việt Nam: 10 chữ số, bắt đầu bằng 0; chấp nhận khoảng trắng, dấu chấm, gạch ngang, và tiền tố `+84`. Viết test dạng bảng (`test.each`).

▼ Lời giải

`examples/src/chapters/ch06/exercise.solution.ts`:

```
export function normalizePhone(input: string): string {
  const digits = input.replace(/[\s.-]/g, '');
  return digits.startsWith('+84') ? `0${digits.slice(3)}` : digits;
}

export function validatePhone(input: string): string | undefined {
  if (!input.trim()) return 'Vui lòng nhập số điện thoại';
  const p = normalizePhone(input);
  if (!/^0\d{9}$/.test(p)) return 'Số điện thoại cần 10 chữ số và bắt đầu bằng 0';
  return undefined;
}
```

```
test.each([
  ['0901 234 567', undefined],
  ['+84 901 234 567', undefined],
  ['090.123.4567', undefined],
  ['12345', 'Số điện thoại cần 10 chữ số và bắt đầu bằng 0'],
  ['', 'Vui lòng nhập số điện thoại'],
])('validatePhone(%p) → %p', (input, expected) => {
  expect(validatePhone(input)).toBe(expected);
});
```

Đây là quy tắc **đơn giản hóa cho bài tập**, không phải quy định chính thức về đầu số của nhà mạng.

Bài 2. Gắn `validatePhone` VÀO `SignUpForm`: thêm ô "Số điện thoại" với `keyboardType="phone-pad"`. Cần sửa những chỗ nào?

▼ Lời giải

1. Thêm `phone: string` VÀO `SignUpValues` VÀ `EMPTY`.
2. Trong `validateSignUp`: `const phoneError = validatePhone(v.phone); if (phoneError) e.phone = phoneError;`

3. Thêm `<TextInput accessibilityLabel="Số điện thoại" keyboardType="phone-pad" textContentType="telephoneNumber" .../>` Và `{show('phone') ? <Text>{errors.phone}</Text> : null}`.

4. Cập nhật test "gửi form hợp lệ" để gõ số điện thoại.

TypeScript sẽ báo lỗi ở mọi chỗ còn thiếu `phone` (ví dụ `EMPTY`) — đó là lợi ích của việc khai báo kiểu `SignUpValues` một lần.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- Handling Text Input: <https://github.com/facebook/react-native-website/blob/main/docs/handling-text-input.md>
- TextInput: <https://github.com/facebook/react-native-website/blob/main/docs/textinput.md>
- KeyboardAvoidingView: <https://github.com/facebook/react-native-website/blob/main/docs/keyboardavoidingview.md>
- Switch: <https://github.com/facebook/react-native-website/blob/main/docs/switch.md>
- RNTL user event API (tài liệu đi kèm gói npm v14): <https://github.com/callstack/react-native-testing-library/blob/main/docs/api/user-event.md>

Chương 7 — Điều hướng (navigation) với Expo Router

Mục tiêu

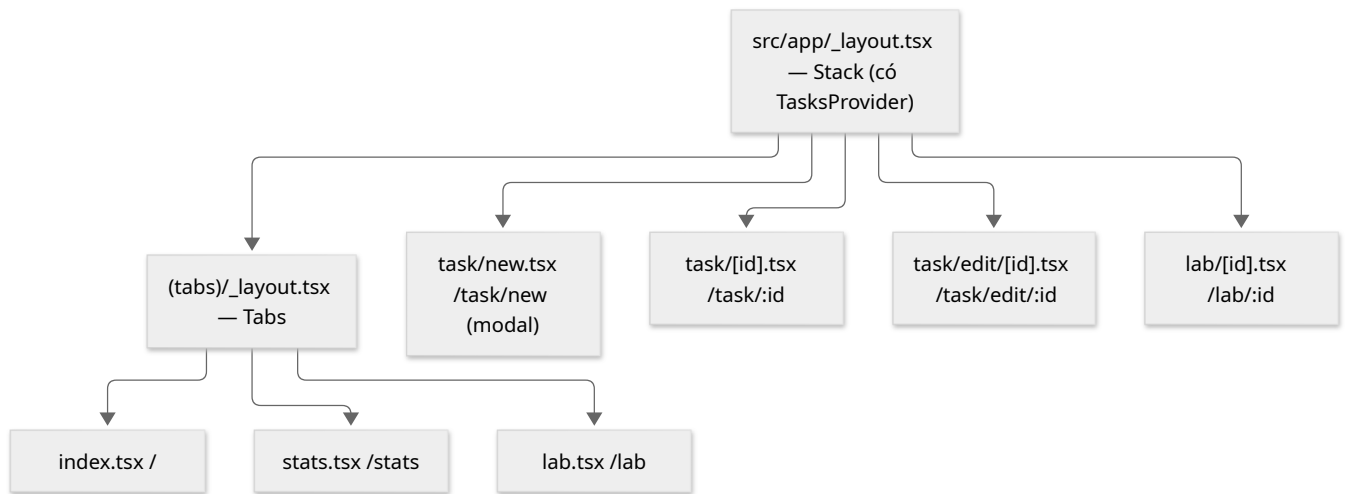
- Hiểu **file-based routing** (định tuyến theo file) của Expo Router.
- Dựng Stack + Tabs + modal + route động `[id]`.
- Điều hướng bằng `<Link>`, `router.push`, `router.back`; đọc tham số bằng `useLocalSearchParams`.
- Viết **test tích hợp** cho điều hướng bằng `renderRouter`.

Giải thích đơn giản

Trong Angular bạn khai báo mảng `routes`. Trong Expo Router, **mỗi file trong `src/app/` là một màn hình**, và đường dẫn file chính là URL:

File	URL
<code>src/app/(tabs)/index.tsx</code>	<code>/</code>
<code>src/app/(tabs)/stats.tsx</code>	<code>/stats</code>
<code>src/app/task/new.tsx</code>	<code>/task/new</code>
<code>src/app/task/[id].tsx</code>	<code>/task/123</code> (tham số <code>id</code>)
<code>src/app/task/edit/[id].tsx</code>	<code>/task/edit/123</code>
<code>src/app/lab/[id].tsx</code>	<code>/lab/ch04</code>

- File `_layout.tsx` định nghĩa **navigator** cho thư mục (Stack = chồng màn hình, Tabs = thanh tab dưới). Giống component chứa `<router-outlet>`.
- Thư mục trong ngoặc `(tabs)` là **route group**: dùng để nhóm, **không** xuất hiện trong URL.
- Code không phải màn hình (component, hook) để **ngoài** `src/app/` (docs Expo Router yêu cầu vậy).



Ví dụ

Layout gốc: Stack + Provider

examples/src/app/_layout.tsx:

```
export default function RootLayout() {
  return (
    <TasksProvider>
      <StatusBar style="auto" />
      <Stack>
        <Stack.Screen name="(tabs)" options={{ headerShown: false }} />
        <Stack.Screen name="task/new" options={{ title: 'Việc mới', presentation: 'modal' }} />
        <Stack.Screen name="task/[id]" options={{ title: 'Chi tiết' }} />
        <Stack.Screen name="task/edit/[id]" options={{ title: 'Sửa việc' }} />
        <Stack.Screen name="lab/[id]" options={{ title: 'Lab' }} />
      </Stack>
    </TasksProvider>
  );
}
```

`presentation: 'modal'` làm màn hình "Việc mới" trượt từ dưới lên trên iPhone.

Tabs

examples/src/app/(tabs)/_layout.tsx:

```
export default function TabsLayout() {
  return (
    <Tabs>
      <Tabs.Screen name="index" options={{ title: 'Việc cần làm', tabBarIcon: () => <Icon
emoji="✅" /> }} />
      <Tabs.Screen name="stats" options={{ title: 'Thống kê', tabBarIcon: () => <Icon
emoji="📊" /> }} />
      <Tabs.Screen name="lab" options={{ title: 'Lab', tabBarIcon: () => <Icon emoji="🔧" />
    }} />
    </Tabs>
  );
}
```

(Dùng emoji làm icon để khỏi thêm thư viện icon.)

Điều hướng

```
// 1) Khai báo: giống [routerLink]
<Link href="/task/new">+</Link>
<Link href={{ pathname: '/lab/[id]', params: { id: item.id } }}>...</Link>

// 2) Mệnh lệnh: giống router.navigate()
const router = useRouter();
router.push(`/task/${id}`);
router.push({ pathname: '/task/edit/[id]', params: { id: task.id } });
router.back();
```

Đọc tham số

examples/src/app/task/[id].tsx:

```
export default function TaskDetailScreen() {
  // Tham số route, giống ActivatedRoute.snapshot.paramMap.get('id')
  const { id } = useLocalSearchParams<{ id: string }>();
  const { tasks, toggleTask, removeTask } = useTasks();
  const task = tasks.find((t) => t.id === id);
  if (!task) return <View style={styles.screen}><Text>Không tìm thấy việc này.</Text></View>;
  return (
    <View style={styles.screen}>
      <Stack.Screen options={{ title: task.title }} />
      { /* ... */ }
    </View>
  );
}
```

<Stack.Screen options> **bên trong** màn hình cho phép đặt tiêu đề động (tên công việc).

Test tích hợp với `renderRouter`

Expo Router có sẵn `expo-router/testing-library`. Nó dựng toàn bộ cây route từ thư mục thật:

```
import { renderRouter } from 'expo-router/testing-library';

async function renderApp(initialUrl: string) {
  const view = renderRouter('./src/app', { initialUrl });
  await view;
  return { get pathname: () => view.get pathname() };
}

it('mở chi tiết khi bấm vào một việc', async () => {
  const app = await renderApp('/');
  const user = userEvent.setup();
  await user.press(await screen.findByLabelText('Mở Đọc chương Flexbox'));
  expect(await screen.findByText('Trạng thái: Chưa xong')).toBeOnTheScreen();
  expect(app.get pathname()).toBe('/task/seed-2');
});
```

Phát hiện khi viết sách (2026-09-28): Với `expo-router@57.0.23` + `RNTL 14.0.1`, `renderRouter()` trả về một **Promise** (vì `render` của `RNTL 14` là `async`) mà các hàm `get pathname()` ... được gắn thêm vào chính Promise đó. Nếu `return view` từ một hàm `async`, JavaScript tự "mở" Promise và các hàm này bị mất. `Matcher expect(screen).toHavePathname()` cũng báo lỗi `screen.get pathname is not a function`. Cách làm trên (giữ biến, `await`, rồi bọc vào object thường) chạy ổn định.

Kết quả (`npx jest --verbose src/__tests__`):

```
PASS src/__tests__/app.routes.test.tsx
App Việc Cần Làm (Expo Router)
  ✓ mở chi tiết khi bấm vào một việc
  ✓ thêm việc mới qua màn hình modal rồi quay về danh sách
  ✓ lọc "Đã xong" chỉ còn việc đã xong
  ✓ màn hình thống kê tính phần trăm
Tab Lab
  ✓ mở ví dụ Flexbox từ danh sách Lab
  ✓ id không tồn tại thì báo lỗi thân thiện
Bài tập Chương 7 và 8
  ✓ sửa tiêu đề qua /task/edit/[id]
  ✓ nút "Xóa việc đã xong" ở tab Thống kê
Xóa việc (bài tập 2, Tập 2 Chương 6)
  ✓ xóa việc sau khi xác nhận trong Alert
```

Đi sâu

Deep link

`app.json` CÓ `"scheme": "rnbookvol1"`. Mọi màn hình đều có URL, nên link `rnbookvol1://task/seed-2` mở thẳng màn hình chi tiết (trong development build / app thật; với Expo Go, URL có dạng `exp://...` — xem tài liệu Linking của Expo). Đây là "universal deep-linking" mà tài liệu Expo Router nhấn mạnh.

`useLocalSearchParams` hay `useGlobalSearchParams`?

Theo tài liệu Expo Router: `useLocalSearchParams` chỉ cập nhật khi URL khớp route **của màn hình này**; `useGlobalSearchParams` cập nhật mỗi khi URL đổi, có thể làm màn hình nền render thừa. Mặc định hãy dùng **local**.

Bảo vệ route (guard)

Angular có `canActivate`. Expo Router có `<Redirect href="/login" />` trong layout và API `Stack.Protected` (xem trang "Protected routes" trong docs). Tập 3 (Security) dùng cách này.

Typed routes

Expo có tùy chọn `experiments.typedRoutes` để TypeScript kiểm tra `href`. Sách **không bật** vì kiểu được sinh ra khi chạy `expo start` (thư mục `.expo/types`, bị `.gitignore`), nên `tsc --noEmit` trên máy mới/CI sẽ lỗi nếu chưa chạy dev server. Bạn có thể bật khi đã quen.

State sống ở đâu khi điều hướng?

`TasksProvider` bọc **ngoài** `<Stack>` trong layout gốc, nên mọi màn hình dùng chung một danh sách. Giống service `providedIn: 'root'`. Nếu đặt Provider trong một màn hình, state sẽ mất khi màn hình bị gỡ.

Lỗi và bẫy thường gặp

- **Đặt component thường trong `src/app/`** → Expo Router coi đó là route.
- **Thiếu `default export`** trong file route → lỗi.
- **Tên trong `<Stack.Screen name>` sai** với file → cảnh báo "Too many screens defined. Route ... is extraneous" (chúng tôi gặp cảnh báo này khi khai báo `lab/[id]` trước khi tạo file).
- **Tham số luôn là chuỗi** (hoặc mảng chuỗi): `id` từ URL không phải number. Tự chuyển kiểu.

- `router.back()` **khi màn hình được mở bằng deep link**: không có màn hình trước để quay lại. Chúng tôi phát hiện lỗi này nhờ test "xóa việc sau khi xác nhận trong Alert" (mở thẳng `/task/seed-2`): React Navigation báo "The action 'GO_BACK' was not handled by any navigator". Cách sửa trong app (`src/lib/navigation.ts`):

```
export function goBackOr(router: Pick<AppRouter, 'canGoBack' | 'back' | 'replace'>, fallback: Href) {
  if (router.canGoBack()) router.back();
  else router.replace(fallback);
}
```

- **Đọc tham số rồi tin tưởng tuyệt đối**: luôn xử lý trường hợp không tìm thấy (xem màn hình "Không tìm thấy việc này" và "Không có ví dụ ...").

Tóm tắt

- File trong `src/app/` = màn hình; `_layout.tsx` = navigator; (group) không vào URL; `[id]` = tham số.
- `<Link>` / `router.push` / `router.back`; `useLocalSearchParams` để đọc tham số.
- Provider đặt ở layout gốc để chia sẻ state giữa các màn hình.
- `renderRouter` cho test tích hợp toàn app.

Bài tập (có lời giải)

Bài 1. Thêm màn hình **Sửa việc** tại `/task/edit/[id]`, mở từ nút "Sửa" ở màn hình chi tiết. Tái sử dụng `TaskForm` với giá trị ban đầu. Sau khi lưu thì quay lại. Viết test tích hợp.

▼ Lời giải

`examples/src/app/task/edit/[id].tsx`:

```
export default function EditTaskScreen() {
  const { id } = useLocalSearchParams<{ id: string }>();
  const { tasks, updateTask } = useTasks();
  const router = useRouter();
  const task = tasks.find((t) => t.id === id);

  if (!task) return <Text>Không tìm thấy việc này.</Text>;

  return (
    <ScrollView keyboardShouldPersistTaps="handled">
      <Stack.Screen options={{ title: 'Sửa việc' }} />
      <TaskForm
        initial={{ title: task.title, note: task.note, priority: task.priority }}
        submitLabel="Cập nhật"
        onSubmit={(input) => {
          updateTask(task.id, input);
          goBackOr(router, { pathname: '/task/[id]', params: { id: task.id } });
        }}
      />
    </ScrollView>
  );
}
```

Nút ở màn hình chi tiết: `router.push({ pathname: '/task/edit/[id]', params: { id: task.id } })`.

Test (trong `src/__tests__/app.routes.test.tsx`): mở `/task/seed-3` → bấm "Sửa" → kiểm tra URL `/task/edit/seed-3` → `user.clear` + `user.type` tiêu đề mới → bấm "Cập nhật" → thấy tiêu đề mới và URL quay về `/task/seed-3`.

Bài 2. Nếu người dùng mở deep link tới một việc không tồn tại, thay vì chữ "Không tìm thấy", hãy tự chuyển về trang chủ. Dùng API nào?

▼ Lời giải

Dùng component `<Redirect>` của Expo Router:

```
import { Redirect } from 'expo-router';
if (!task) return <Redirect href="/" />;
```

Lời giải được test riêng trong `examples/src/chapters/ch07/redirect.solution.test.tsx`, dùng **mock routes** (truyền object thay cho thư mục vào `renderRouter`):

```
function Detail() {
  const { id } = useLocalSearchParams<{ id: string }>();
  if (!KNOWN.includes(id)) return <Redirect href="/" />;
  return <Text>Chi tiết {id}</Text>;
}

const view = renderRouter(
  { index: () => <Text>Trang chủ</Text>, 'task/[id]': Detail },
  { initialUrl: '/task/khong-co' },
);
await view;
expect(await screen.findByText('Trang chủ')).toBeOnTheScreen();
expect(view.getPathname()).toBe('/');
```

Kết quả: ✓ deep link tới id không tồn tại thì quay về trang chủ, ✓ id hợp lệ thì ở lại màn hình chi tiết. App mẫu vẫn giữ màn hình "Không tìm thấy" vì thân thiện hơn khi học.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- Expo Router — Core concepts: <https://github.com/expo/expo/blob/main/docs/pages/router/basics/core-concepts.mdx>
- Expo Router — Notation (group, dynamic route): <https://github.com/expo/expo/blob/main/docs/pages/router/basics/notation.mdx>
- Expo Router — Navigation layouts: <https://github.com/expo/expo/blob/main/docs/pages/router/basics/navigation-layouts.mdx>
- Expo Router — URL parameters: <https://github.com/expo/expo/blob/main/docs/pages/router/reference/url-parameters.mdx>
- Expo Router — Redirects: <https://github.com/expo/expo/blob/main/docs/pages/router/reference/redirects.mdx>
- Expo Router — Protected routes: <https://github.com/expo/expo/blob/main/docs/pages/router/advanced/protected.mdx>
- Expo Router — Testing: <https://github.com/expo/expo/blob/main/docs/pages/router/reference/testing.mdx>
- Expo Router — Typed routes: <https://github.com/expo/expo/blob/main/docs/pages/router/reference/typed-routes.mdx>

Chương 8 — App mẫu Tập 1: "Việc Cần Làm"

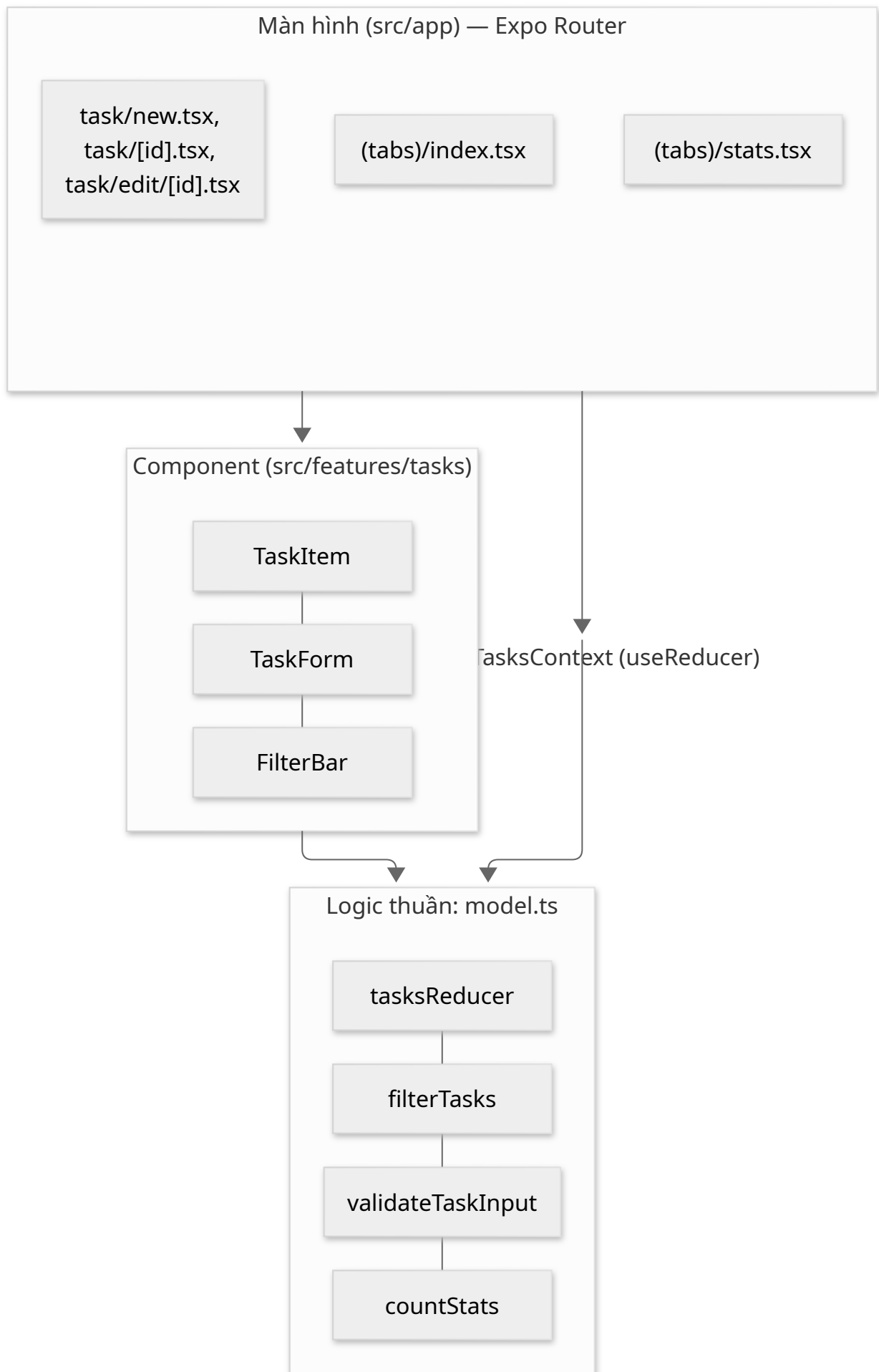
Mục tiêu

Ghép mọi thứ của Tập 1 thành một app hoàn chỉnh chạy trong Expo Go:

- Tabs (Việc cần làm / Thống kê / Lab), Stack, modal, route động.
- Danh sách có tìm kiếm + bộ lọc, form có validate, màn hình chi tiết, sửa, xóa.
- State dùng chung bằng `useReducer` + Context.
- Test 3 tầng: logic thuần → component → toàn app (router).

Giải thích đơn giản

App được chia 3 lớp, giống cách bạn chia Angular app thành model/service/component:



Quy tắc: **logic thuần không import React Native** → test nhanh, không cần render.

Ví dụ

Chạy app (lệnh chính xác)

```
cd books/react-native/vol1-co-ban/examples
npm ci # cài đúng phiên bản trong package-lock.json
npx expo login # một lần; cùng tài khoản với Expo Go trên iPhone
npx expo start # quét QR bằng Camera của iPhone
```

Kiểm tra chất lượng:

```
npm run typecheck # tsc --noEmit
npm run lint # eslint .
npm test # jest --ci
npm run export:ios # (tùy chọn) bundle Hermes cho iOS để chắc chắn build được
```

Reducer — trái tim của app

examples/src/features/tasks/model.ts:

```
export type TaskAction =
  | { type: 'add'; task: Task }
  | { type: 'toggle'; id: string }
  | { type: 'remove'; id: string }
  | { type: 'update'; id: string; changes: Partial<TaskInput> }
  | { type: 'clearDone' }; // lời giải bài tập Chương 8

export function tasksReducer(state: Task[], action: TaskAction): Task[] {
  switch (action.type) {
    case 'add':
      return [action.task, ...state];
    case 'toggle':
      return state.map((t) => (t.id === action.id ? { ...t, done: !t.done } : t));
    case 'remove':
      return state.filter((t) => t.id !== action.id);
    case 'update':
      return state.map((t) => (t.id === action.id ? { ...t, ...action.changes } : t));
    case 'clearDone':
      return state.filter((t) => !t.done);
  }
}
```

TypeScript kiểm tra `switch` đủ mọi `type` (discriminated union — union có trường phân biệt). Nếu bạn quên một case, hàm có thể trả `undefined` và `tsc` báo lỗi vì kiểu trả về là `Task[]`.

Context — "service" dùng chung

`examples/src/features/tasks/TasksContext.tsx` bọc `useReducer` và đưa ra các hàm `addTask`, `toggleTask`, `removeTask`, `updateTask`, `clearDone`. Màn hình chỉ gọi `const { tasks, toggleTask } = useTasks();` — giống `inject(TasksService)`.

Màn hình danh sách

```
const visible = useMemo(() => filterTasks(tasks, filter, query), [tasks, filter, query]);
const openTask = useCallback((id: string) => router.push(`/task/${id}`), [router]);

<FlatList
  data={visible}
  keyExtractor={(t) => t.id}
  renderItem={({ item }) => <TaskItem task={item} onToggle={toggleTask} onOpen={openTask} />}
  ListEmptyComponent={<Text style={styles.empty}>Không có việc nào.</Text>}
/>
```

`TaskItem` bọc `memo`, `openTask` bọc `useCallback`, `toggleTask` đến từ `Context` (được `useMemo` giữ ổn định cho tới khi `tasks` đổi) → khi gõ tìm kiếm, các dòng không đổi sẽ không render lại vô ích.

Kết quả kiểm tra toàn bộ Tập 1 (chạy thật 2026-09-28)

```
bash books/react-native/scripts/check-all.sh vol1-co-ban
```

Trích output:

```
--- typecheck
--- lint
--- test
Test Suites: 16 passed, 16 total
Tests:       57 passed, 57 total
--- export web
web bundle OK
--- export ios
ios bundle OK
ALL CHECKS PASSED: vol1-co-ban
```

(`typecheck` và `lint` không in gì khi không có lỗi.)

Chạy trên Expo Go: NOT RUN (không có iPhone trong sandbox). Bằng chứng thay thế: test tích hợp toàn app bằng `renderRouter` và bundle Hermes iOS được tạo thành công.

Đi sâu

Test 3 tầng

Tầng	File	Kiểm tra gì	Tốc độ
Logic thuần	features/tasks/model.test.ts	reducer, lọc, validate, thống kê	Rất nhanh
Component	features/tasks/TaskForm.test.tsx	form hiện lỗi, gửi dữ liệu	Nhanh
Toàn app	__tests__/app.routes.test.tsx	điều hướng thật giữa các màn hình	Chậm hơn

Giống "kim tự tháp test": nhiều test thuần, ít test tích hợp.

Accessibility để test và cho người dùng

Mọi nút có `accessibilityRole` + nhãn; checkbox có `accessibilityState={{ checked }}`; bộ lọc dùng `role="tab" + selected`; mức ưu tiên dùng `role="radio"`. Nhờ đó test viết được `getByRole('tab', { name: 'Đã xong' })` và VoiceOver đọc đúng.

Hạn chế có chủ ý (sẽ làm ở Tập 2-3)

- Dữ liệu **mất khi tắt app** → Tập 2: lưu offline (AsyncStorage/SQLite).
- Chưa có animation khi thêm/xóa → Tập 2: Reanimated.
- Chưa tối ưu danh sách rất dài → Tập 3: FlashList, React Compiler.

Lỗi và bẫy thường gặp

- Provider đặt sai chỗ** (trong một màn hình) → mỗi màn hình có danh sách riêng.
- Quên quay lại** sau khi lưu form modal → modal không đóng.
- router.back() khi mở bằng deep link** → không có màn hình trước, lỗi "The action 'GO_BACK' was not handled by any navigator". App dùng `goBackOr(router, fallback)` trong `src/lib/navigation.ts` (xem Chương 7).
- Alert trong test**: xem kỹ thuật `jest.spyOn(Alert, 'alert')` ở Tập 2 Chương 6; test "xóa việc sau khi xác nhận trong Alert" đã có trong `app.routes.test.tsx`.
- So sánh id kiểu number với string** từ URL → không tìm thấy task.
- Tạo id bằng Math.random() trong render** → id đổi mỗi lần render. Tạo id **một lần** khi thêm (`createTask`).

Tóm tắt

- App = logic thuần + state (Context/useReducer) + component + màn hình (Expo Router).
- Chia lớp giúp test dễ: 57 test chạy trong vài giây.
- Các chương sau sẽ thêm lưu trữ, animation, API thật, bảo mật và phát hành.

Bài tập (có lời giải)

Bài 1. Thêm nút "Xóa N việc đã xong" ở tab Thống kê. Nút bị vô hiệu khi N = 0.

▼ Lời giải

1. Thêm action `{ type: 'clearDone' }` VÀO `TaskAction` và case trong reducer (xem code ở trên).
2. Thêm `clearDone: () => dispatch({ type: 'clearDone' })` VÀO `TasksContext`.
3. Trong `src/app/(tabs)/stats.tsx`:

```
<AppButton title={`Xóa ${stats.done} việc đã xong`} variant="danger" disabled={stats.done === 0} onPress={clearDone} />
```

Test (đã có trong dự án):

```
clearDone (bài tập Chương 8)  
✓ xóa mọi việc đã xong  
Bài tập Chương 7 và 8  
✓ nút "Xóa việc đã xong" ở tab Thống kê
```

Bài 2. Thêm sắp xếp theo mức ưu tiên (Cao → Thấp) trong `filterTasks` mà **không** làm hỏng test cũ. Bạn sẽ đặt logic ở đâu và test thế nào?

▼ Lời giải

Đặt trong `model.ts` như một hàm thuần riêng, rồi gọi sau `filterTasks` ở màn hình:

```
const RANK: Record<Priority, number> = { high: 0, medium: 1, low: 2 };  
export function sortByPriority(tasks: Task[]): Task[] {  
  return [...tasks].sort((a, b) => RANK[a.priority] - RANK[b.priority] ||  
    b.createdAt - a.createdAt);  
}
```

Test thuần: `expect(sortByPriority(SEED_TASKS).map(t => t.id)).toEqual(['seed-1', 'seed-2', 'seed-3'])`. Tách hàm riêng thay vì sửa `filterTasks` giúp test cũ không đổi (nguyên tắc single responsibility). Hàm này có trong `model.ts` và test "sortByPriority (bài tập 2 Chương 8)" trong `model.test.ts`.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- React — useReducer / Context (mã nguồn react.dev): <https://github.com/reactjs/react.dev/blob/main/src/content/reference/react/useReducer.md>
- React — Scaling up with Reducer and Context: <https://github.com/reactjs/react.dev/blob/main/src/content/learn/scaling-up-with-reducer-and-context.md>
- Expo Router — Core concepts: <https://github.com/expo/expo/blob/main/docs/pages/router/basics/core-concepts.mdx>
- React Native — Accessibility: <https://github.com/facebook/react-native-website/blob/main/docs/accessibility.md>
- Optimizing FlatList Configuration: <https://github.com/facebook/react-native-website/blob/main/docs/optimizing-flatlist-configuration.md>

STACK — Phiên bản và thư viện dùng cho cả bộ sách

Tất cả phiên bản dưới đây được kiểm tra ngày **2026-09-28** bằng `npm view`, bằng file `bundledNativeModules.json` bên trong gói `expo@57.0.25`, và bằng mã nguồn tài liệu chính thức trên GitHub. Trang `reactnative.dev`, `docs.expo.dev`, `expo.dev` bị chặn trong sandbox, nên chúng tôi đọc **mã nguồn** của các trang đó trên GitHub (xem mục "Nguồn tham khảo").

1. Tóm tắt nhanh (TL;DR)

Thành phần	Phiên bản ghim (pinned)	Vì sao
Node.js	22.22.2 (LTS)	RN 0.85+ yêu cầu Node $\geq 20.19.4$; Node 22 là Active LTS
Expo SDK	57 (<code>expo@57.0.25</code>)	SDK mới nhất ổn định (<code>npm latest</code> = 57.0.25). Expo Go chỉ chạy SDK mới nhất
React Native	0.86.3	Phiên bản mà Expo SDK 57 đóng gói (<code>bundledNativeModules.json</code>)
React	19.2.3	Theo Expo SDK 57
TypeScript	6.0.3	Theo template <code>expo-template-default@57.0.27</code>
Điều hướng	<code>expo-router@57.0.23</code>	Mặc định của Expo, file-based routing
Test	<code>jest@29.7.0</code> , <code>jest-expo@57.0.5</code> , <code>@testing-library/react-native@14.0.1</code> , <code>test-renderer@1.3.0</code>	Preset chính thức của Expo
Lint	<code>eslint@9.39.5</code> , <code>eslint-config-expo@57.0.2</code>	Cấu hình flat của Expo
JS engine	Hermes V1 (mặc định từ RN 0.84)	Không cần cấu hình
Kiến trúc	New Architecture (bắt buộc từ RN 0.82)	Không thể tắt nữa

Chạy trên Expo Go (iPhone): các app trong sách dùng SDK 57. Expo Go trên App Store chỉ hỗ trợ **một** SDK (SDK mới nhất). Xem mục 4 để biết mức độ đã kiểm chứng.

2. Phiên bản React Native mới nhất và New Architecture

- `npm view react-native version` → **0.87.1** (bản mới nhất trên npm, 2026-09-28). RN 0.87.0 phát hành 2026-08-11.
- Nhưng Expo SDK 57 dùng **0.86.3**. Chúng ta theo Expo, vì Expo Go chỉ chạy đúng phiên bản RN mà SDK đóng gói. **Quyết định:** dùng 0.86.3, không dùng 0.87.

- Mốc quan trọng (từ blog chính thức trong repo react-native-website):
 - 0.76 (2024-10): New Architecture thành mặc định.
 - **0.82 (2025-10-08): "the first React Native that runs entirely on the New Architecture"** — đặt `newArchEnabled=false` sẽ bị bỏ qua. Expo SDK 54 / RN 0.81 là bản cuối còn Legacy Architecture.
 - **0.84 (2026-02-11): Hermes V1 là JavaScript engine mặc định.**
 - 0.85 (2026-04-07): Jest preset tách ra `@react-native/jest-preset`; bỏ Node < 20.19.4; Xóa `StyleSheet.absoluteFillObject`.
 - **0.86 (2026-06-11):** không có breaking change cho người dùng; sửa edge-to-edge Android 15+; repo React Native chuyển từ tổ chức `facebook` sang `react` trên GitHub.
 - 0.87 (2026-08-11): Strict TypeScript API mặc định (chưa có trong SDK 57).
- Expo docs (`guides/new-architecture.mdx`): "Expo Go only supports the New Architecture."

3. Expo hay React Native CLI?

Tài liệu chính thức của React Native (`docs/getting-started.md`, repo `react-native-website`) viết: "if you're building a new app with React Native, we recommend using a Framework", và giới thiệu Expo là "a production-grade React Native Framework". Lệnh khuyên dùng: `npx create-expo-app@latest`.

Tiêu chí	Expo (framework)	React Native CLI (không framework)
Bắt đầu	<code>npx create-expo-app@latest</code>	Cần Xcode/Android Studio ngay
Chạy trên iPhone không cần Mac	Có, qua Expo Go	Không (cần Xcode để build)
Routing	Expo Router (file-based) có sẵn	Tự chọn thư viện
Native module tùy ý	Cần development build (không dùng Expo Go)	Có
Build & phát hành	EAS Build / EAS Submit (cloud) hoặc local	Tự làm bằng Xcode/Gradle

Quyết định: cả bộ sách dùng **Expo** (managed + Continuous Native Generation). Lý do: (1) docs chính thức khuyên dùng framework; (2) Nobin test bằng Expo Go trên iPhone; (3) Tập 3 vẫn dạy native module và build thật qua development build / EAS.

4. Expo SDK hiện tại và Expo Go trên iPhone

- `npm view expo dist-tags` (2026-09-28): `latest` = **57.0.25**, `next` = 58.0.0-preview.8 (SDK 58 đang beta). Ngày phát hành 57.0.0 trên npm: 2026-06-30.
- Expo docs (`workflow/upgrading-expo-sdk-walkthrough.mdx`): Expo Go "only supports the latest SDK version and previous versions are no longer supported". Và

(`develop/development-builds/faq.md`): "Each build of Expo Go supports one SDK version. The project and Expo Go SDK versions must match."

- Expo docs (`get-started/start-developing.md`): trên iPhone thật, Expo Go chỉ mở project khi **Expo CLI và Expo Go đăng nhập cùng một tài khoản Expo**: chạy `npx expo login` trên máy tính, rồi đăng nhập trong Expo Go (biểu tượng tài khoản góc trên bên phải).
- **UNVERIFIED (một phần)**: "Expo Go trên App Store hiện hỗ trợ SDK 57". Chúng tôi không mở được `apps.apple.com` hay `expo.dev/changelog` (bị chặn). Bằng chứng phụ: (a) kết quả WebSearch nói Expo Go iOS 57.0.9 lên App Store ngày 2 tháng 9; (b) PR công khai ngày 2026-09-20 trên GitHub (YoniRipp/beme#371) nói "Expo Go ships only the latest SDK — currently 57". Nobin nên mở Expo Go → Settings để xem "Supported SDK".
- **Cảnh báo**: khi SDK 58 ổn định, Expo Go trên App Store sẽ chuyển sang SDK 58 và các app SDK 57 sẽ không mở được nữa. Khi đó chạy `npx expo install expo@^58.0.0 --fix` trong từng thư mục `examples/` (xem README).

5. Thư viện cho từng tập (tất cả có trong Expo Go SDK 57)

Cột "Nguồn phiên bản": BNM = `bundledNativeModules.json` trong `expo@57.0.25`; `npm` = `npm view <pkg> version`. `npx expo install` luôn chọn đúng bản BNM.

Tập	Thư viện	Phiên bản	Nguồn phiên bản	Dùng để
1	expo-router	57.0.23	BNM	Điều hướng
1	react-native-safe-area-context	5.7.0	BNM	Vùng an toàn (tai thỏ)
1	react-native-screens	4.26.x	BNM	Màn hình native
1	react-native-web, react-dom	0.21.x, 19.2.3	BNM	Chạy expo export --platform web để kiểm tra
2	zustand	5.0.15	npm (JS thuần)	State toàn cục
2	@tanstack/react-query	5.104.0	npm (JS thuần)	Cache dữ liệu server
2	@react-native-async-storage/async-storage	2.2.0	BNM	Key-value lưu trữ
2	expo-sqlite	~57.0.3	BNM	Cơ sở dữ liệu SQLite
2	react-native-reanimated / react-native-worklets	4.5.1 / 0.10.1	BNM	Animation
2	expo-haptics, expo-location, expo-image-picker	~57.0.x	BNM	Device APIs
3	expo-secure-store	~57.0.4	BNM	Lưu bí mật (Keychain/Keystore)
3	@shopify/flash-list	2.0.2	BNM	Danh sách hiệu năng cao
3	@sentry/react-native	~7.11.0	BNM	Monitoring (chỉ giới thiệu; cần DSN)
3	eas-cli	24.8.0	npm	Build/Submit (chạy bằng npx eas-cli@latest)

Phiên bản chính xác đã cài nằm trong `package.json` + `package-lock.json` của từng `vol*/examples/` (ghim exact, không dùng `^/~`).

So sánh thư viện state (Tập 2)

Repo	Stars	Commit gần nhất	License	Hữu ích	Ngày kiểm tra
https://github.com/pmndrs/zustand	58.8k	2026-08-25	MIT	API nhỏ, giống "service có state" của Angular	2026-09-28
https://github.com/reduxjs/redux-toolkit	11.2k	2026-09-28	MIT	Chuẩn doanh nghiệp, giống NgRx	2026-09-28
https://github.com/pmndrs/jotai	21.3k	2026-09-08	MIT	State dạng atom, gần giống Signals	2026-09-28
https://github.com/TanStack/query	50.4k	2026-09-28	MIT	Server state (cache, retry)	2026-09-28

Quyết định: Zustand cho client state (ít boilerplate, dễ test) + TanStack Query cho server state. Redux Toolkit và Jotai được giới thiệu so sánh trong chương.

6. Công cụ trong sandbox (dùng để kiểm tra sách)

Công cụ	Phiên bản
Node / npm	22.22.2 / 10.9.7
pandoc	3.1.3
Playwright Chromium (in PDF)	playwright 1.56.1

Lưu ý khi cài trong sandbox: `npx expo install` gọi API của expo.dev (bị chặn), nên dùng `EXPO_OFFLINE=1 npx expo install <pkg>` — Expo CLI khi đó dùng bảng phiên bản có sẵn trong gói expo.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-28:

- npm registry: `npm view expo dist-tags`, `npm view expo time`, `npm view react-native version`, `npm view react-native time`, `npm view expo-template-default@sdk-57` — <https://www.npmjs.com/package/expo>
- `bundledNativeModules.json` trong gói expo@57.0.25 (tải bằng `npm pack expo@57.0.25`).
- Expo docs source, commit 99d902f (2026-09-28): <https://github.com/expo/expo/tree/main/docs/pages>
 - <https://github.com/expo/expo/blob/main/docs/pages/get-started/start-developing.mdx>
 - <https://github.com/expo/expo/blob/main/docs/pages/workflow/upgrading-expo-sdk-walkthrough.mdx>
 - <https://github.com/expo/expo/blob/main/docs/pages/develop/development-builds/faq.mdx>
 - <https://github.com/expo/expo/blob/main/docs/pages/guides/new-architecture.mdx>
 - <https://github.com/expo/expo/blob/main/docs/pages/versions/v57.0.0/sdk/securestore.mdx>
- React Native website source, commit f5d7ce0 (2026-09-27):
 - <https://github.com/facebook/react-native-website/blob/main/docs/getting-started.md>
 - <https://github.com/facebook/react-native-website/blob/main/website/blog/2025-10-08-react-native-0.82.mdx>
 - <https://github.com/facebook/react-native-website/blob/main/website/blog/2026-02-11-react-native-0.84.mdx>
 - <https://github.com/facebook/react-native-website/blob/main/website/blog/2026-04-07-react-native-0.85.mdx>
 - <https://github.com/facebook/react-native-website/blob/main/website/blog/2026-06-11-react-native-0.86.mdx>

- <https://github.com/facebook/react-native-website/blob/main/website/blog/2026-08-11-react-native-0.87.mdx>
- Bảng chứng phụ (secondary) cho Expo Go SDK 57 trên App Store: <https://github.com/YoniRipp/beme/pull/371>
- Trang repo GitHub (stars, license): <https://github.com/pmndrs/zustand>, <https://github.com/reduxjs/redux-toolkit>, <https://github.com/pmndrs/jotai>, <https://github.com/TanStack/query>
- Bị chặn, KHÔNG mở được: <https://expo.dev/changelog/sdk-57>, <https://apps.apple.com/us/app/expo-go/id982107779>, <https://reactnative.dev>, <https://docs.expo.dev>

GLOSSARY — Thuật ngữ React Native (Anh → Việt)

Thuật ngữ (English)	Giải thích tiếng Việt	Gần giống trong Angular
React Native (RN)	Framework viết app iOS/Android bằng React; vẽ view native thật	Ionic/Capacitor (nhưng Ionic dùng WebView)
Expo	Framework cho RN: CLI, router; thư viện module native, dịch vụ build	Angular CLI + thư viện chính thức
Expo SDK	Bộ phiên bản Expo đi cùng một phiên bản RN (sách dùng SDK 57 ↔ RN 0.86)	Phiên bản Angular major
Expo Go	App trên App Store để chạy thử project mà không cần build	—
Development build	Bản app "Expo Go của riêng bạn", có native module tùy ý	—
EAS (Expo Application Services)	Dịch vụ cloud: build, submit, update	CI/CD dịch vụ
Metro	Bundler (công cụ đóng gói JS) của RN	esbuild/webpack trong Angular CLI
Hermes	JavaScript engine chạy trên điện thoại; V1 mặc định từ RN 0.84	V8 trong trình duyệt
Fast Refresh	Cập nhật app ngay khi lưu file, giữ state	HMR (<code>ng serve --hmr</code>)
New Architecture	Kiến trúc mới của RN: JSI + Fabric + TurboModules; bắt buộc từ RN 0.82	—
JSI (JavaScript Interface)	Lớp C++ cho JS gọi native trực tiếp, không qua "bridge"	—
Fabric	Hệ thống render mới của RN	Renderer
TurboModules	Native module thế hệ mới, tải lười (lazy)	—
Codegen	Sinh code C++/native từ type TypeScript	—
Component	Hàm trả về JSX	<code>@Component</code>
JSX	Cú pháp giống HTML viết trong TypeScript	Template
Props	Tham số truyền vào component (chỉ đọc)	<code>@Input()</code>
Callback prop	Hàm truyền xuống để con báo sự kiện lên	<code>@Output()</code>
State	Dữ liệu thay đổi được của component (<code>useState</code>)	<code>signal()</code> / field của class
Hook	Hàm <code>use...</code> dùng tính năng React trong component hàm	Lifecycle hooks + <code>inject()</code>
Effect (<code>useEffect</code>)	Side effect sau khi render, có hàm dọn dẹp	<code>ngOnInit</code> + <code>ngOnDestroy</code> , <code>effect()</code>
Memo (<code>useMemo</code> , <code>memo</code>)	Ghi nhớ kết quả/Component để tránh tính/render lại	<code>computed()</code> , <code>OnPush</code>
Context	Truyền giá trị xuống cây component không qua props	Dependency Injection
Controlled component	Input mà giá trị nằm trong state React	<code>[(ngModel)]</code> , <code>FormControl</code>
Pressable	Component bắt sự kiện chạm	<code>(click)</code> trên <code><button></code>

Thuật ngữ (English)	Giải thích tiếng Việt	Gần giống trong Angular
FlatList / SectionList	Danh sách ảo hóa (virtualized), chỉ render dòng đang thấy	<code>cdk-virtual-scroll</code>
Virtualization	Chỉ render phần tử đang nằm trên màn hình	Virtual scroll
Flexbox	Hệ thống bố cục; RN mặc định <code>flexDirection: 'column'</code>	CSS Flexbox
StyleSheet	API tạo style cho component	SCSS của component
Safe area	Vùng không bị tai thỏ/thanh home che	—
Expo Router	Router theo file (file-based routing) cho Expo	Angular Router
Layout (<code>_layout.tsx</code>)	File định nghĩa navigator (Stack, Tabs) cho một thư mục	Component chứa <code><router-outlet></code>
Stack / Tabs	Kiểu điều hướng: chồng màn hình / thanh tab dưới	—
Deep link	Link mở thẳng vào một màn hình trong app	URL route
Accessibility (a11y)	Khả năng tiếp cận: VoiceOver, vai trò, nhãn	ARIA
jest-expo	Preset Jest của Expo	Karma/Jest preset Angular
RNTL	React Native Testing Library — test theo cách người dùng dùng app	TestBed + Testing Library
Continuous Native Generation (CNG)	Thư mục <code>ios/</code> và <code>android/</code> được sinh ra từ <code>app.json</code> , không sửa tay	—
Config plugin	Plugin sửa cấu hình native khi sinh <code>ios/ / android/</code>	Angular schematics (gần giống)